

Pengujian Kapabilitas Deteksi *Anti-Spyware Mode Stealth* dalam Pengumpulan Data Intelijen Siber

Proposal Tugas Akhir

Oleh

**Audra Zelvania Putri Harjanto
18222106**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Desember 2025**

LEMBAR PENGESAHAN

Pengujian Kapabilitas Deteksi *Anti-Spyware Mode Stealth* dalam Pengumpulan Data Intelijen Siber

Proposal Tugas Akhir

Oleh

**Audra Zelvania Putri Harjanto
18222106**

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Proposal Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 5 Desember 2025

Pembimbing

Prof. Dr. Ing. Ir. Suhardi, M.T.

NIP. 196312111990011002.

DAFTAR ISI

DAFTAR GAMBAR	iv
DAFTAR TABEL	v
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan	3
I.4 Batasan Masalah	3
I.5 Metodologi	4
II STUDI LITERATUR	6
II.1 <i>Spyware</i>	6
II.1.1 Definisi dan Karakteristik <i>Spyware</i>	6
II.1.2 <i>Spyware</i> Mode <i>Stealth</i>	7
II.2 <i>Data Crawling</i>	8
II.2.1 Definisi <i>Data Crawling</i>	8
II.2.2 Jenis Data Target <i>Crawling</i>	8
II.2.3 Teknik dan Mekanisme <i>Data Crawling</i>	9
II.3 <i>Exfiltration</i>	10
II.3.1 Definisi dan Tujuan <i>Exfiltration</i>	10
II.3.2 Metode <i>Exfiltration</i>	11
II.4 <i>Anti-Spyware</i>	12
II.4.1 Definisi <i>Anti-Spyware</i>	12
II.4.2 Mekanisme Deteksi <i>Anti-Spyware</i>	12
II.5 Studi Sebelumnya	14
III ANALISIS MASALAH	16
III.1 Analisis Kondisi Saat Ini	16
III.1.1 Tantangan <i>Data Crawling</i> dan Eksfiltrasi <i>Spyware</i>	16
III.1.2 <i>Gap Analysis</i>	17
III.2 Analisis Kebutuhan	18
III.2.1 Identifikasi Masalah Pengguna	18
III.2.2 Kebutuhan Fungsional	19
III.2.3 Kebutuhan Non-Fungsional	20
III.3 Analisis Pemilihan Solusi	21

III.3.1 Alternatif Solusi	21
IV DESAIN KONSEP SOLUSI	27
IV.1 Tahapan Desain	27
IV.1.1 Rancangan Modul <i>Mission</i>	27
IV.1.1.1 Arsitektur Modul <i>Mission</i>	27
IV.1.1.2 Target Data dan Lingkungan Operasi	30
IV.1.1.3 Alur Kerja Modul <i>Mission</i>	31
IV.1.2 Rancangan Modul <i>Exfiltration</i>	33
IV.1.2.1 Arsitektur Modul <i>Exfiltration</i>	34
IV.1.2.2 Target Output <i>Exfiltration</i>	36
IV.1.2.3 Mekanisme Pengiriman Data	36
IV.1.2.4 Rancangan Lingkungan Pengujian	38
IV.2 Hasil Desain	39
V RENCANA SELANJUTNYA	41
V.1 Rencana Implementasi	41
V.1.1 Rencana Implementasi Modul <i>Mission</i>	41
V.1.2 Rencana Implementasi Modul <i>Exfiltration</i>	42
V.2 Rencana Evaluasi	43
V.2.1 Parameter dan Metrik Evaluasi	43
V.2.2 Kriteria Keberhasilan	44
V.3 Analisis Risiko	45

DAFTAR GAMBAR

I.1	Framework <i>Design Research Methodology</i> (2009)	4
II.1	Data <i>Exfiltration</i>	10
III.1	Alur Kerja <i>Data Crawling</i> dan Eksfiltrasi <i>Spyware</i> Saat Ini	16
IV.1	Diagram Blok Arsitektur Modul <i>Mission</i>	28
IV.2	<i>Flowchart</i> Alur Kerja Modul <i>Mission</i>	32
IV.3	Diagram Blok Arsitektur Modul <i>Exfiltration</i>	34
IV.4	<i>Flowchart</i> Mekanisme Pengiriman Data pada Modul <i>Exfiltration</i> . .	37
IV.5	Diagram Hasil Desain Arsitektur Sistem	39

DAFTAR TABEL

II.1	Studi Sebelumnya	14
III.1	<i>Gap Analysis</i>	18
III.2	Kebutuhan Fungsional	20
III.3	Kebutuhan Non-Fungsional	21
III.4	Masalah dan Dampak	22
III.5	Analisis Alternatif Solusi	23
III.6	<i>Weighted Scoring Model</i>	26
IV.1	Teknologi yang Digunakan pada Setiap Komponen Modul <i>Mission</i> .	30
IV.2	Tabel IV.2 Target Data	30
IV.3	Target <i>Output Exfiltration</i>	36
IV.4	Struktur <i>Payload</i>	36
IV.5	Konfigurasi Lingkungan Pengujian	38
V.1	Rencana Implementasi Modul <i>Mission</i>	42
V.2	Rencana Implementasi Modul <i>Exfiltration</i>	43
V.3	Parameter dan Metrik Evaluasi	44
V.4	Kriteria Keberhasilan	45
V.5	Analisis Risiko dan Mitigasi	46

BAB I

PENDAHULUAN

I.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat dapat membawa manfaat besar bagi suatu organisasi maupun individu. Namun, di sisi lain juga dapat meningkatkan kerentanan terhadap serangan siber. Menurut *World Economic Forum*, serangan siber dan kerentanannya berada di peringkat empat besar risiko utama yang dapat mengancam stabilitas global. Indonesia menjadi salah satu negara yang paling sering menjadi target serangan siber secara global. Berdasarkan laporan Badan Siber dan Sandi Negara (BSSN) pada Desember 2024, Indonesia menempati posisi lima besar negara paling sering diserang secara siber. Serangan utama yang terjadi meliputi *ransomware*, *phishing*, dan *Distributed Denial of Service* (DDoS), yang menasar infrastruktur penting nasional, institusi pemerintahan, dan sektor bisnis. Pada semester pertama tahun 2025, tercatat sebanyak 133,4 juta serangan siber di Indonesia, dengan serangan jenis *Generic Protocol Command Decode* mendominasi sebanyak 68,37%. Selain itu, serangan DDoS dan *botnet* Mirai juga menjadi ancaman utama yang memengaruhi kestabilan dan keamanan sistem informasi di dalam negeri.

Salah satu ancaman siber yang semakin mengkhawatirkan adalah serangan yang berasal dari *spyware*. *Spyware* merupakan salah satu jenis *malware* yang dirancang khusus untuk mengumpulkan data sensitif pengguna tanpa sepengetahuan mereka. Ada berbagai jenis *spyware* yang umum ditemukan, antara lain *keylogger* yang merekam aktivitas pengetikan keyboard, *adware* yang menampilkan iklan secara agresif, serta *trojan* yang memungkinkan akses jarak jauh ke perangkat korban. *Spyware* modern semakin canggih dengan kemampuan menyembunyikan jejak dan beroperasi secara tersembunyi agar sulit dideteksi oleh sistem keamanan standar. Selain *spyware* tipe *file-based*, terdapat juga *spyware fileless* yang bekerja langsung di me-

mori (RAM) sehingga mampu menghindari teknik deteksi berbasis *signature*. Salah satu contoh *spyware* yang berbahaya adalah Pegasus, yang memiliki kemampuan infiltrasi tanpa interaksi pengguna (*zero click exploit*) dan dapat mengakses berbagai jenis data pribadi serta mengendalikan perangkat secara *remote* tanpa terdeteksi (Reddy dkk. 2024).

Dalam pengumpulan data intelijen siber, *data crawling* merupakan proses otomatis yang memungkinkan pengambilan informasi dari berbagai sumber dalam skala besar dan sistematis. Proses ini melibatkan eksploitasi dan pengambilan data secara terus-menerus dari target, yang dapat berupa situs web, server, hingga perangkat yang terinfeksi *malware*. Data yang dikumpulkan kemudian diproses secara lebih mendalam untuk memperoleh informasi relevan yang dapat digunakan dalam analisis ancaman (Verma dkk. 2021). Dalam konteks *spyware* mode *stealth*, *data crawling* difasilitasi oleh *spyware* yang mampu menyusup tanpa terdeteksi dan secara otomatis mengumpulkan data pengguna secara diam-diam.

Setelah data berhasil dikumpulkan, tahap eksfiltrasi menjadi proses kritis berikutnya, di mana data hasil pencurian dikirimkan secara tersembunyi ke server penyerang. Teknik eksfiltrasi yang digunakan dapat berupa trafik data terenkripsi, pemecahan data menjadi bagian kecil yang dikirim secara berkala untuk menghindari deteksi, atau penyamaran data dalam komunikasi normal jaringan.

Untuk menghadapi ancaman yang ditimbulkan oleh *spyware*, upaya proteksi menjadi sangat penting dalam menjaga keamanan sistem dan data. *Anti-spyware* adalah perangkat lunak yang dirancang untuk mendeteksi, mencegah, dan menghapus *spyware*. Dalam menghadapi *spyware* modern, terutama yang menggunakan teknik *stealth* seperti *fileless malware* yang berjalan langsung di memori, metode deteksi tradisional berbasis *signature* dianggap kurang efektif.

Dalam penelitian ini, fokus utama terdapat pada tahap *mission*, yaitu proses pengumpulan data (*data crawling*) dan pengiriman data hasil pengintaian secara tersembunyi (*eksfiltrasi*) oleh *spyware* mode *stealth*. Pelaksanaan *data crawling* dan eksfiltrasi ini dilakukan untuk menguji kapabilitas *anti-spyware* dalam mendeteksi aktivitas pengumpulan dan pengiriman data yang dilakukan secara diam-diam oleh *spyware*. Dengan demikian, proses ini bertujuan untuk mengetahui efektivitas suatu sistem keamanan dalam menangkal aksi pencurian data intelijen siber yang tersembunyi dan sulit dideteksi.

I.2 Rumusan Masalah

Berikut adalah rumusan masalah yang dibahas dalam tugas akhir:

1. Bagaimana cara mengimplementasikan mekanisme pengambilan data (*crawling*) dan eksfiltrasi oleh *spyware* mode *stealth* dari sistem target tanpa terdeteksi oleh sistem keamanan?
2. Bagaimana kinerja perangkat *anti-spyware* dalam mendeteksi aktivitas *data crawling* dan eksfiltrasi yang dilakukan oleh *spyware* mode *stealth*?

I.3 Tujuan

Berikut adalah tujuan dari pengerjaan tugas akhir:

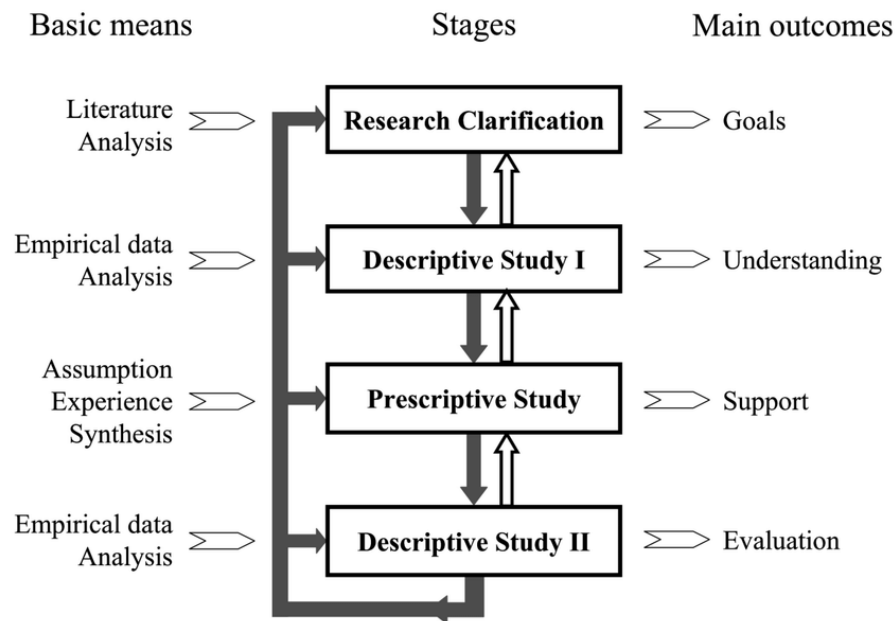
1. Mengimplementasikan mekanisme pengambilan data dan eksfiltrasi oleh *spyware* mode *stealth* dari sistem target, serta menguji bagaimana proses tersebut dapat berjalan tanpa terdeteksi oleh sistem keamanan.
2. Mengevaluasi kinerja perangkat *anti-spyware* dalam mendeteksi aktivitas *data crawling* dan eksfiltrasi yang dilakukan oleh *spyware* mode *stealth*, sehingga dapat diperoleh rekomendasi terkait efektivitas serta peningkatan sistem keamanan terhadap ancaman tersebut.

I.4 Batasan Masalah

Batasan masalah dalam pelaksanaan tugas akhir adalah sebagai berikut:

1. Penelitian ini hanya berfokus pada pengujian kapabilitas deteksi *anti-spyware* terhadap aktivitas *data crawling* dan eksfiltrasi yang dilakukan oleh *spyware* mode *stealth* pada sistem operasi desktop.
2. Evaluasi hanya mencakup perangkat *anti-spyware* yang dipilih sesuai kriteria penelitian, tidak membandingkan seluruh produk *anti-spyware* yang ada.
3. Aktivitas *data crawling* dan eksfiltrasi yang diujikan dilakukan dalam batasan jaringan dan topologi tertentu sesuai rancangan penelitian.
4. Tugas akhir ini dikerjakan secara kelompok dengan anggota penelitian sebagai berikut:
 - Nathaniel Liady
 - M. Kasyfil Aziz
 - Audra Zelvania Putri Harjanto
 - Khayla Belva Annandira

I.5 Metodologi



Gambar I.1 Framework *Design Research Methodology* (2009)

Penelitian ini mengadopsi kerangka *Design Research Methodology* (DRM) yang dipublikasikan oleh Lucienne Blessing dan Amaresh Chakrabarti pada tahun 2009. Pendekatan ini memungkinkan penelitian dilakukan secara terstruktur mulai dari klarifikasi masalah hingga tahap evaluasi solusi. Dalam konteks tugas akhir ini, DRM digunakan untuk merancang dan mengevaluasi proses *data crawling* dan *data exfiltration* dalam *spyware mode stealth*. DRM terdiri dari empat tahapan utama, yaitu *Research Clarification*, *Descriptive Study I*, *Prescriptive Study*, dan *Descriptive Study II*.

1. ***Research Clarification (RC)*** berfokus pada pemahaman awal mengenai permasalahan yang diteliti, ruang lingkup, serta tujuan penelitian. Pada tahap ini dirumuskan *research questions* mengenai bagaimana mekanisme *data crawling* dan eksfiltrasi dapat diimplementasikan dalam mode *stealth* serta bagaimana perangkat *anti-spyware* merespons aktivitas tersebut. Tahap RC menjadi dasar konseptual dalam merumuskan arah penelitian dan merancang kriteria keberhasilan evaluasi.
2. ***Descriptive Study I (DS-I)*** bertujuan memperdalam pemahaman mendalam mengenai konteks, konsep, dan faktor-faktor yang berkaitan dengan permasalahan penelitian. Pada tahap ini dilakukan studi literatur untuk mengidentifikasi bagaimana proses yang diteliti dijelaskan dalam penelitian sebelumnya.

Hasil analisis ini kemudian digunakan untuk menyusun *reference model*, yaitu gambaran awal mengenai bagaimana proses yang diteliti dipahami menurut pengetahuan yang ada saat ini, yang nantinya menjadi dasar bagi perancangan solusi pada tahap berikutnya.

3. ***Prescriptive Study (PS)*** berfokus pada perancangan solusi atau pendekatan yang akan dikembangkan untuk menjawab permasalahan penelitian. Berdasarkan pemahaman yang diperoleh dari tahap sebelumnya, pada tahap ini disusun rancangan konseptual yang menjelaskan bagaimana solusi tersebut diharapkan bekerja. Tahap ini menghasilkan *impact model*, yaitu gambaran mengenai pengaruh yang diharapkan dari solusi yang dirancang terhadap proses atau sistem yang diteliti. Rancangan yang dihasilkan kemudian menjadi dasar bagi implementasi dan pengujian pada tahap berikutnya.
4. ***Descriptive Study II (DS-II)*** digunakan untuk mengevaluasi solusi yang telah dirancang pada tahap sebelumnya. Pada tahap ini, solusi tersebut diimplementasikan dan diuji dalam kondisi yang direncanakan untuk melihat sejauh mana hasil aktual sesuai dengan harapan yang digambarkan dalam *impact model*. Proses evaluasi ini bertujuan mengamati kinerja solusi, mengidentifikasi keberhasilan maupun keterbatasannya, serta menganalisis faktor-faktor yang memengaruhi hasil tersebut. Hasil dari DS-II digunakan untuk menilai efektivitas solusi secara menyeluruh dan memberikan dasar bagi penyempurnaan atau pengembangan studi lebih lanjut.

BAB II

STUDI LITERATUR

II.1 *Spyware*

II.1.1 Definisi dan Karakteristik *Spyware*

Menurut Reddy dkk. (2024), *spyware* secara umum didefinisikan sebagai salah satu bentuk *malicious software (malware)* yang dirancang untuk memantau, merekam, dan mengirimkan informasi sensitif pengguna tanpa sepengetahuan maupun persetujuan pemilik perangkat. Kondisi ini menimbulkan ancaman serius terhadap kerahasiaan data dan privasi digital. Dalam ranah *cyber espionage*, *spyware* modern telah berevolusi menjadi alat intelijen siber berkapabilitas tinggi. Pegasus, misalnya, merupakan *advanced spyware* yang mampu menginfeksi *smartphone* melalui berbagai vektor serangan dan mengekstraksi hampir seluruh jenis data, termasuk komunikasi terenkripsi, sehingga ancamannya mencakup individu, organisasi, hingga level negara (Kareem 2024). Dengan kemampuan pengawasan jangka panjang, pelacakan lokasi, serta pemantauan aktivitas secara *real time*, *spyware* menempati posisi penting dalam ekosistem ancaman keamanan modern.

Secara karakteristik, *spyware* biasanya memiliki kemampuan untuk beroperasi secara tersembunyi (*stealth*), bertahan lama (*persistence*), serta melakukan komunikasi jarak jauh dengan server pengendali (*command and control*) untuk mengirimkan data yang berhasil dikumpulkan (Salih dan Mohammed 2020). Pada platform Android, *spyware* kerap disisipkan ke dalam aplikasi palsu atau aplikasi yang tampak fungsional, lalu meminta berbagai *permission* sensitif, seperti akses kontak, pesan, lokasi, kamera, dan mikrofon sebagai bagian dari *social engineering*.

Kareem (2024) menambahkan bahwa *spyware* tingkat lanjut dapat memanfaatkan kerentanan *zero day* dan mekanisme serangan *zero click* sehingga mampu menginfeksi perangkat tanpa perlu interaksi pengguna, sekaligus menghapus jejak untuk

mengurangi kemungkinan analisis forensik. Dari sisi perilaku, *spyware* umumnya menargetkan berbagai jenis data, mulai dari kredensial, pesan instan, email, daftar kontak, log panggilan, riwayat penelusuran, hingga data sensor seperti lokasi GPS dan rekaman audio atau video. Beberapa implementasi juga menyertakan fungsi *keylogging*, pengambilan *screenshot*, dan pengumpulan informasi perangkat yang kemudian dimanfaatkan untuk *profiling* target serta perencanaan serangan lanjutan.

II.1.2 *Spyware Mode Stealth*

Elghaly (2024) menjelaskan bahwa *spyware mode stealth* merujuk pada kelas *spyware* yang dirancang untuk beroperasi secara terselubung dengan meminimalkan jejak artefak forensik, sehingga sulit terdeteksi oleh pengguna maupun solusi keamanan seperti antivirus dan *Endpoint Detection and Response* (EDR). Salah satu ciri utama mode *stealth* adalah pemanfaatan teknik *fileless* dan *living off the land*, yaitu menjalankan *payload* langsung di memori serta menyalahgunakan komponen bawaan sistem seperti PowerShell, WMI, dan *task scheduler*, sehingga tidak meninggalkan berkas *executable* mencolok di disk. Dari sisi teknik, (Elghaly 2024) mendeskripsikan bahwa mode *stealth* sering bergantung pada beberapa mekanisme: eksekusi *in memory*, *code obfuscation*, penggunaan komunikasi terenkripsi, serta pemanfaatan *persistence* yang terselubung melalui *registry*, *scheduled task*, atau *startup folder* yang tidak secara eksplisit menunjukkan nama *malware*.

Spyware canggih seperti Pegasus menggabungkan kemampuan penyamaran proses, enkripsi komunikasi, dan mekanisme *self-destruct* untuk mempertahankan *persistence* sekaligus menghapus bukti ketika upaya analisis terdeteksi (Kareem 2024). Pada kasus Pegasus, aspek *stealth* mencakup penggunaan *exploit zero click*, penyamaran sebagai proses sistem yang sah, komunikasi *command and control* (C2) terenkripsi, serta kemampuan untuk menyesuaikan diri dengan lingkungan target dan menghilang ketika indikator analisis terdeteksi. *Spyware* ini juga memanfaatkan berbagai vektor infeksi seperti *phishing*, *exploit chain*, *watering hole attacks*, dan *network injection* sehingga proses infeksi dapat terjadi tanpa interaksi pengguna, memperkuat karakteristik *stealth* baik pada fase infeksi maupun fase operasi pengumpulan data.

Sudhakar dan Kumar (2020) menjelaskan bahwa *fileless malware* dirancang untuk beroperasi langsung di memori dengan memanfaatkan komponen bawaan sistem seperti PowerShell dan Windows Management Instrumentation (WMI), sehingga hampir tidak meninggalkan berkas tradisional di disk. Karakteristik ini menjadikan *fileless malware* sulit dideteksi oleh solusi keamanan berbasis *signature* dan selaras

dengan tujuan operasi *stealth* karena aktivitas berbahaya berbaur dengan aktivitas administratif yang sah.

II.2 Data Crawling

II.2.1 Definisi Data Crawling

Data *crawling* adalah proses otomatis yang dilakukan untuk mengumpulkan data dari berbagai sumber, terutama dari internet atau sistem digital, dengan cara menjelajahi dan mengambil konten secara sistematis menggunakan perangkat lunak khusus yang disebut *crawler*. Proses ini memungkinkan pengumpulan informasi yang kemudian dapat dianalisis lebih lanjut untuk berbagai keperluan, termasuk dalam ranah keamanan siber dan intelijen digital (Salih dan Mohammed 2020). Dalam konteks siber, *crawling* digunakan *spyware* sebagai metode pengumpulan data korban secara tersembunyi dan berkelanjutan.

Kareem (2024) menambahkan bahwa teknik *crawling* pada *spyware* modern seperti Pegasus diperluas dengan penggunaan metode eksfiltrasi data untuk memperluas jangkauan pengumpulan informasi, baik dari perangkat korban maupun sumber eksternal, sehingga efektivitas pengintaian digital semakin meningkat.

Menurut Jegan, Rajavarman, dan Geetha (2024), *crawling* dipahami sebagai proses penelusuran data yang dilakukan secara otomatis, terstruktur, dan berulang untuk menjangkau berbagai informasi digital. Sistem *crawling* dirancang untuk navigasi dan menelusuri struktur data, baik dalam bentuk jaringan, tautan, maupun elemen-elemen sistem lain, untuk mengekstraksi konten serta mengidentifikasi pola atau informasi penting dari sumber data yang kompleks. Konsep ini sejalan dengan pemanfaatan *crawling* dalam perangkat lunak berbahaya, di mana prinsip *systematic traversal* yang digunakan pada *web crawling* juga diterapkan pada lingkungan perangkat korban, misalnya dengan menelusuri direktori, memindai berkas, membaca metadata, atau mengakses *cache* aplikasi.

II.2.2 Jenis Data Target Crawling

Menurut penelitian yang dilakukan Salih dan Mohammed (2020), *spyware* Android yang mereka kembangkan dan uji disembunyikan di dalam aplikasi permainan palsu dengan tujuan memperoleh berbagai informasi korban, seperti kontak, pesan, panggilan, akun, serta lokasi geografis melalui Wi-Fi atau SIM, 3G, 4G, dan LTE, serta informasi pribadi lain yang tersimpan di perangkat. Selain itu, *spyware* ter-

sebut juga dapat menampilkan daftar korban, mengambil seluruh kontak, membaca pesan masuk dan keluar, serta melihat *call log*.

Kareem (2024) menjelaskan bahwa Pegasus, setelah berhasil menginfeksi *smartphone*, mampu mengakses berbagai jenis data sensitif dari perangkat target, termasuk pesan, email, panggilan, riwayat penelusuran, data dari aplikasi media sosial, serta data dari aplikasi pesan terenkripsi. Kareem (2024) juga menyebutkan bahwa Pegasus dapat mengaktifkan mikrofon dan kamera untuk merekam audio dan video, melacak lokasi secara *real time*, menangkap penekanan tombol, serta mengumpulkan berkas dan pengenalan unik perangkat.

II.2.3 Teknik dan Mekanisme *Data Crawling*

Teknik dan mekanisme *data crawling* dalam konteks *spyware* dan keamanan siber menggambarkan bagaimana sistem berbahaya maupun *security monitoring tools* mengotomatisasi proses pengumpulan informasi dari perangkat korban. Menurut Salih dan Mohammed (2020), teknik *crawling* pada *spyware* Android diimplementasikan dengan menyisipkan kode *spyware* ke dalam aplikasi permainan palsu yang berjalan seperti aplikasi biasa namun di latar belakang mengeksekusi fungsi pengambilan data. Saat aplikasi dibuka, *spyware* secara otomatis mengumpulkan kontak, pesan, *call log*, akun, dan lokasi, kemudian mengirimkannya ke *REST API* berbasis PHP yang menyimpan data tersebut pada basis data MySQL sehingga operator dapat mengaksesnya melalui *control panel* tanpa memerlukan interaksi tambahan dengan korban.

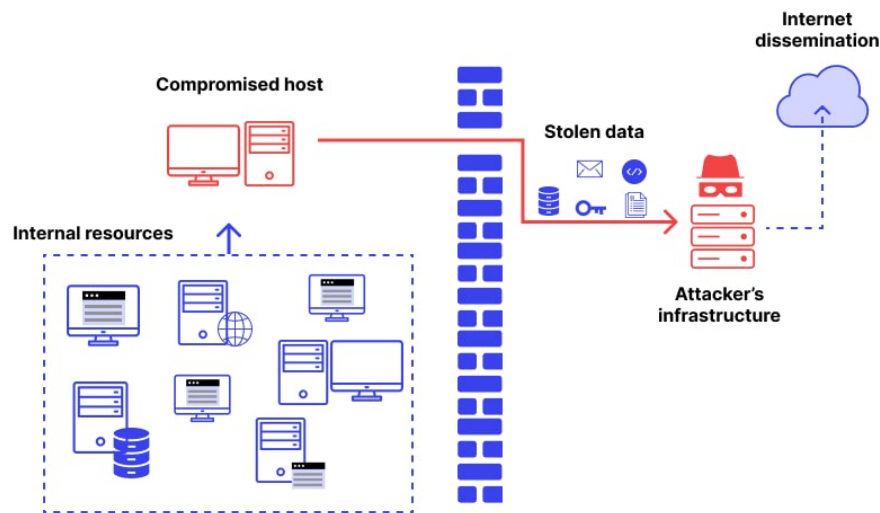
Pada *spyware* Pegasus, setelah tahap infeksi, *spyware* menjalankan modul untuk mengekstraksi berbagai jenis data dari perangkat target. Pada bagian *data collection and exfiltration*, Kareem (2024) menjelaskan bahwa Pegasus mengakses pesan instan, media sosial, riwayat penelusuran, sandi tersimpan, kalender, catatan, pengenalan perangkat, serta berkas lokal maupun *cloud*. Data tersebut kemudian dienkripsi dan dikirimkan ke server *command and control* (C2) sebagai mekanisme eksfiltrasi yang terotomatisasi dan berulang.

Penelitian yang dilakukan Jegan, Rajavarman, dan Geetha (2024) secara spesifik membahas mekanisme *web crawling* pada lingkungan *dark web*, yaitu proses penelusuran otomatis terhadap halaman dan tautan untuk mengumpulkan informasi seperti kredensial yang terekspos dan diskusi terkait serangan siber. Mekanisme *data crawling* dalam sistem mereka dimulai dari pemberian kata kunci atau URL awal yang kemudian digunakan *crawler* untuk menelusuri *dark web* dan mengeks-

traksi informasi penting seperti *compromised credentials*. *Crawler* secara otomatis mengirim *HTTP request*, mengambil halaman, mengekstrak konten, kemudian memanfaatkan *natural language processing* (NLP) dan *machine learning* (ML) untuk menginterpretasi data tidak terstruktur serta mendeteksi pola anomali yang mengindikasikan potensi ancaman.

II.3 Exfiltration

II.3.1 Definisi dan Tujuan Exfiltration



Gambar II.1 Data Exfiltration

Definisi dan tujuan eksfiltrasi dalam konteks *spyware* dan keamanan siber merujuk pada proses pengambilan dan pengiriman keluar data dari sistem atau perangkat target secara tidak sah oleh pihak penyerang. Reddy dkk. (2024) menjelaskan eksfiltrasi sebagai salah satu aktivitas utama *payload spyware*, yaitu pengambilan dan pengiriman keluar data sensitif secara tidak sah setelah *spyware* berhasil dijalankan di dalam sistem korban. *Spyware* pada perangkat yang telah terkompromi mengakses berbagai sumber daya internal, mengumpulkan data sensitif, kemudian mengirimkannya keluar melewati batas jaringan menuju infrastruktur penyerang. Data yang telah dieksfiltrasi tersebut selanjutnya dapat dimanfaatkan sesuai tujuan penyerang.

Menurut King dkk. (2021), *data exfiltration* didefinisikan sebagai proses pemindahan data secara tidak sah dari dalam jaringan atau sistem organisasi ke pihak luar

melalui berbagai media dan protokol komunikasi. Eksfiltrasi merupakan tahap akhir dari banyak insiden keamanan, ketika pelaku yang telah memperoleh akses berusaha mengeluarkan informasi bernilai tinggi. Tujuan utama *exfiltration* adalah mengumpulkan dan mengirimkan data bernilai ke infrastruktur penyerang untuk kemudian dimonetisasi, dimanfaatkan dalam spionase, atau digunakan sebagai *leverage* dalam pemerasan. Proses eksfiltrasi dijaga kerahasiaannya dengan menyamarkan lalu lintas keluar agar terlihat seperti trafik normal, misalnya dengan memanfaatkan protokol umum serta pola komunikasi yang menyerupai aktivitas sah di jaringan.

II.3.2 Metode *Exfiltration*

King dkk. (2021) menjelaskan bahwa metode *data exfiltration* dapat dikategorikan menjadi tiga kelompok utama, yaitu *content-based*, *header-based*, dan *meta-based methods*. Dalam studi tersebut, diimplementasikan beberapa teknik eksfiltrasi, seperti penyisipan data pada citra digital (*image steganography*), pemanfaatan HTTP 404 untuk menyamarkan transfer data pada *web logs*, manipulasi nilai *Time To Live* (TTL), manipulasi *IP options field*, serta pemanfaatan *e-mail header* untuk membawa *payload* eksfiltrasi sebelum kemudian diuji apakah serangan tersebut dapat dideteksi oleh IDS Snort.

Ullah dkk. (2018) mengulas berbagai vektor serangan eksternal yang digunakan untuk eksfiltrasi data, antara lain *web-based exfiltration* melalui HTTP/HTTPS, eksfiltrasi berbasis *e-mail*, *DNS tunneling*, protokol *file transfer*, layanan *cloud storage* dan *file sharing*, serta berbagai *covert channels*. Penulis menekankan bahwa penggunaan protokol dan layanan yang umum digunakan organisasi, seperti HTTP/HTTPS dan layanan *cloud*, membuat lalu lintas eksfiltrasi sulit dibedakan dari trafik sah, sehingga setiap vektor dibahas bersamaan dengan ringkasan *countermeasures* yang dapat diterapkan.

Penelitian oleh Zhan dkk. (2022) menunjukkan bahwa penyerang dapat memanfaatkan *DNS over HTTPS* (DoH) sebagai kanal eksfiltrasi dengan menyembunyikan permintaan DNS di dalam trafik HTTPS terenkripsi. Studi ini menganalisis karakteristik trafik DoH, meliputi *TLS fingerprint* dan fitur-fitur aliran (*flow-based features*) yang berbeda dari pola penggunaan normal, serta mengusulkan metode deteksi berbasis *machine learning* untuk mengidentifikasi *DoH tunneling*.

II.4 *Anti-Spyware*

II.4.1 Definisi *Anti-Spyware*

Baldota, Shelar, dan Parbhane (2025) menyatakan bahwa *anti-spyware* merupakan perangkat lunak atau mekanisme keamanan yang dirancang khusus untuk mengidentifikasi, menghapus, dan mencegah aktivitas *spyware* dalam sistem komputer maupun perangkat bergerak. Fungsi utama *anti-spyware* tidak hanya mendeteksi dan menghapus program *spyware* yang berbahaya, tetapi juga melindungi privasi pengguna serta mencegah pencurian data secara diam-diam oleh aplikasi tidak sah. Perangkat *anti-spyware* dilengkapi dengan berbagai teknik, mulai dari *signature-based detection*, *behavior-based detection*, *heuristics*, hingga metode berbasis *machine learning* untuk memperluas jangkauan dan efektivitas deteksi terhadap beragam varian *spyware* yang terus berkembang. Selain peran deteksi dan penghapusan, *anti-spyware* juga berfungsi memperkuat lapisan keamanan digital dengan mencegah eksekusi maupun inisiasi aktivitas *spyware* lebih lanjut di perangkat korban. Dengan integrasi ke pengamanan *endpoint* dan pengawasan aplikasi, *anti-spyware* menjadi bagian penting dari strategi proteksi data modern.

Qabalin, Naser, dan Alkasassbeh (2022) menekankan bahwa modul deteksi *spyware* merupakan inti dari mekanisme *anti-spyware* dalam ekosistem perangkat Android. Dengan kemampuan membedakan aplikasi *spyware* dari aplikasi resmi, *anti-spyware* menjaga perlindungan perangkat agar terbebas dari ancaman pengumpulan data tersembunyi oleh program jahat. Sistem *anti-spyware* dirancang untuk mengenali ciri-ciri aplikasi yang melakukan pemantauan, pengumpulan informasi, perekaman aktivitas, atau perubahan pengaturan tanpa persetujuan pengguna sebagai bagian dari fungsi pengintaian.

II.4.2 Mekanisme Deteksi *Anti-Spyware*

Untuk mengidentifikasi dan mencegah aktivitas pengintaian yang dilakukan *spyware*, berbagai mekanisme deteksi telah dikembangkan dalam solusi keamanan modern. Setiap mekanisme berupaya mengenali keberadaan *spyware* melalui pendekatan yang berbeda, baik dengan mencocokkan ciri khas *malware* yang sudah diketahui, menganalisis perilaku mencurigakan selama program berjalan, maupun menilai karakteristik yang tidak biasa berdasarkan aturan dan skor tertentu. Beberapa penelitian juga mengusulkan pemanfaatan teknik pembelajaran mesin untuk meningkatkan akurasi deteksi terhadap varian *spyware* baru yang sulit dikenali dengan metode tradisional. Secara umum, mekanisme deteksi *anti-spyware* dapat dikelom-

pokkan ke dalam *signature-based detection*, *behavior-based* dan *heuristic detection*, pendekatan *machine learning-based detection*, serta deteksi berbasis trafik jaringan.

1. *Signature Based Detection*

Menurut Baldota, Shelar, dan Parbhane (2025), banyak solusi *anti-spyware* tradisional mengandalkan *signature-based detection*, yaitu mencocokkan pola biner atau ciri khas *spyware* yang sudah diketahui dengan berkas atau proses yang sedang diperiksa. Pendekatan ini efektif untuk *spyware* yang sudah dikenal, tetapi kurang mampu mendeteksi varian baru atau *spyware* yang di-*obfuscate* karena tidak memiliki *signature* yang tersimpan di basis data.

2. *Behavior Based dan Heuristic Detection*

Behavior-based detection memonitor aktivitas program, seperti akses berkas, perubahan *registry*, koneksi jaringan, dan interaksi dengan API sistem, untuk mengidentifikasi pola perilaku yang menyerupai *spyware*, misalnya pengambilan data sensitif atau aktivitas tersembunyi. Mekanisme ini sering dipadukan dengan *heuristic detection*, yang menggunakan aturan atau *scoring* untuk menilai apakah kombinasi perilaku tertentu cukup mencurigakan untuk dikategorikan sebagai ancaman (Baldota, Shelar, dan Parbhane 2025).

3. *Machine Learning Based Detection*

Pendekatan modern banyak memanfaatkan algoritma *machine learning* untuk membedakan *spyware* dari aplikasi resmi berdasarkan fitur statis maupun dinamis (Baldota, Shelar, dan Parbhane 2025). Penelitian yang dilakukan oleh Abualhaj dkk. (2024) menunjukkan bahwa model *decision tree* yang dioptimalkan mampu mengklasifikasikan sampel *spyware* dengan akurasi tinggi pada dataset *malware*, sehingga dapat dijadikan modul deteksi pada sistem *anti-spyware*. Sementara itu, penelitian oleh Wang dkk. (2025) mengembangkan deteksi *Trojan spyware* Android dengan menggabungkan fitur statis seperti *permission* dan metadata aplikasi serta fitur dinamis dari perilaku *runtime*, kemudian melatih model *machine learning* untuk membedakan *spyware* dari aplikasi *benign*.

4. Deteksi Berbasis Trafik Jaringan

Metode SNDMI yang diperkenalkan dalam penelitian oleh Peng dkk. (2024) memfokuskan deteksi *spyware* pada analisis pola trafik jaringan, termasuk penggunaan trafik *inducement* dan fitur-fitur *flow* untuk melatih model LightGBM. Pendekatan ini menempatkan modul deteksi di level jaringan sehingga *spyware* dapat diidentifikasi melalui pola komunikasi ke server eksternal tanpa harus selalu bergantung pada inspeksi berkas atau proses pada *endpoint*.

II.5 Studi Sebelumnya

Studi sebelumnya diperlukan untuk memahami perkembangan penelitian terkait *data crawling*, mekanisme pengumpulan data oleh *spyware*, serta berbagai teknik ek-sfiltrasi dan deteksi yang digunakan dalam keamanan siber. Kajian ini memberikan gambaran mengenai pendekatan, metode, serta keterbatasan penelitian terdahulu yang menjadi dasar dalam merumuskan ruang lingkup dan kontribusi penelitian ini.

Tabel II.1 Studi Sebelumnya

No	Judul Penelitian (Penulis)	Metode	Hasil Utama	Keterbatasan
1	<i>Design and Implementation of an Android Spyware Application Using a Fake Game</i> (Salih dan Mohammed 2020)	Perancangan sistem dan eksperimen	Mengembangkan <i>spyware</i> Android yang melakukan <i>data crawling</i> terhadap kontak, pesan, <i>log</i> panggilan, akun, dan lokasi, serta meng-ekstraksinya secara otomatis ke server <i>backend</i> melalui REST API	Berfokus pada An-droid saja, tidak membahas mekanis-me <i>stealth</i> tingkat lanjut atau deteksi oleh <i>anti-spyware</i> .
2	<i>Cyber Threat Intelligence through Automated Dark Web Crawling</i> (Jegan, Rajavarman, dan Geetha 2024)	Perancangan sistem dan eksperimen	Mengimplementasikan <i>web crawling</i> oto-matis pada <i>dark web</i> dengan ekstraksi konten, pemetaan tautan, dan deteksi kredensial yang ter-ekspos menggunakan <i>machine learning</i> .	Fokus pada <i>crawling</i> di <i>dark web</i> untuk CTI, bukan <i>crawling</i> internal pada perang-kat korban seperti <i>spyware</i> .

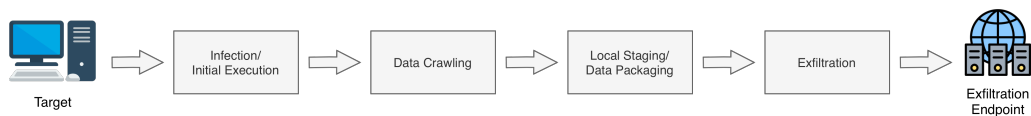
No	Judul Penelitian (Penulis)	Metode	Hasil Utama	Keterbatasan
3	<i>Stealthy Data Collection and Exfiltration in Pegasus Spyware</i> (Kareem 2024)	Studi literatur teknis dan analisis arsitektur	Mendeskripsikan mekanisme <i>data collection</i> dan <i>auto-exfiltration spyware</i> Pegasus, mencakup akses pesan, email, media sosial, <i>keylogging</i> , rekaman audio/video, lokasi <i>real-time</i> , serta pengiriman terenkripsi ke C2.	Tidak menyertakan implementasi uji empiris; sebagian informasi berasal dari laporan investigatif, bukan eksperimen langsung.
4	<i>An Emerging Threat: Fileless Malware – A Survey and Research Challenges</i> (Sudhakar dan Kumar 2020)	Studi literatur	Menjelaskan teknik penyembunyian dan eksfiltrasi <i>fileless</i> , termasuk penggunaan <i>PowerShell</i> , <i>registry abuse</i> , dan <i>memory-resident payload</i> .	Tidak membahas implementasi <i>crawling</i> secara spesifik atau eksfiltrasi berbasis protokol tertentu.

BAB III

ANALISIS MASALAH

III.1 Analisis Kondisi Saat Ini

Bagian ini menjelaskan kondisi saat ini terkait bagaimana *spyware* tradisional melakukan proses mulai dari infeksi, pengambilan data, hingga eksfiltrasi. Alur kerja *spyware* saat ini digunakan sebagai dasar untuk memahami pola kerja *spyware* yang ada saat ini.



Gambar III.1 Alur Kerja *Data Crawling* dan Eksfiltrasi *Spyware* Saat Ini

Gambar III.1 menggambarkan alur kerja umum *spyware* dalam melakukan pengambilan dan pengiriman data dari perangkat target. Proses dimulai dari tahap infeksi atau eksekusi awal, ketika *spyware* berhasil dijalankan di dalam sistem target dan menempatkan dirinya. Setelah aktif, *spyware* memulai tahap *data crawling*, yaitu proses pengumpulan data secara otomatis dan sistematis terhadap berbagai informasi sensitif. Data yang terkumpul kemudian tidak langsung dikirimkan, melainkan diproses terlebih dahulu melalui *local staging*, misalnya dengan kompresi, enkripsi, atau pemecahan data ke dalam paket kecil agar lebih mudah disamarkan. Tahap selanjutnya adalah eksfiltrasi, yaitu proses pengiriman data yang telah dipaketkan menuju infrastruktur penyerang melalui kanal komunikasi tertentu. Setelah itu data mencapai *exfiltration endpoint*.

III.1.1 Tantangan *Data Crawling* dan Eksfiltrasi *Spyware*

Pada bagian ini dibahas berbagai hambatan yang muncul ketika *spyware* berusaha menjalankan kedua proses tersebut tanpa terdeteksi oleh sistem keamanan.

1. Aktivitas *crawling* yang *high noise* dan mudah terpantau
Proses *crawling* sering melibatkan akses file, direktori, dan API secara intensif dan berulang. Aktivitas ini menghasilkan pola I/O yang mencolok dan dapat terdeteksi sebagai anomali oleh sistem keamanan. Ketika tidak ada mekanisme untuk mengurangi jejak atau menyesuaikan frekuensi akses, pola *crawling* menjadi mudah dibedakan dari perilaku aplikasi normal.
2. Eksfiltrasi melalui kanal standar yang mudah dipantau
Pengiriman data sering dilakukan melalui protokol umum seperti HTTP/HTTPS, email, FTP, atau DNS sederhana. Meskipun protokol tersebut merupakan bagian dari trafik normal dalam suatu jaringan, pola eksfiltrasi yang tidak disamarkan atau tidak menyerupai komunikasi aplikasi sah tetap dapat terdeteksi oleh *anti-spyware*, terutama ketika frekuensi, ukuran, atau struktur paketnya berbeda dari penggunaan wajar.
3. Pola eksfiltrasi yang kasar atau *bulk transfer*.
Data kerap dikirim dalam jumlah besar sekaligus atau pada interval waktu tetap. Pola pengiriman yang seragam dan tidak dinamis seperti ini mudah dikenali sebagai perilaku mencurigakan, karena berbeda dari arus komunikasi normal yang biasanya lebih bervariasi.
4. Risiko deteksi dari pola akses yang intensif.
Crawling yang melibatkan pembacaan *file*, pemindaian direktori, atau interaksi API dalam frekuensi tinggi dapat menimbulkan pola aktivitas yang berbeda dari aplikasi normal. Sistem keamanan berbasis perilaku dapat mengenali anomali ini sebagai indikasi aktivitas tidak sah.

III.1.2 Gap Analysis

Pada bagian ini disajikan *gap analysis* yang mengidentifikasi perbedaan antara kondisi saat ini (*As-Is*) dan kondisi ideal (*To-Be*) terkait kemampuan pengumpulan data (*crawling*) dan proses eksfiltrasi. Tabel III.1 merangkum domain masalah utama, observasi kondisi saat ini, kesenjangan yang teridentifikasi, serta kondisi ideal yang menjadi sasaran perbaikan.

Tabel III.1 *Gap Analysis*

Domain Masalah	Kondisi Saat Ini (<i>As-Is</i>)	Kesenjangan (<i>Gap</i>)	Kondisi Ideal (<i>To-Be</i>)
Aktivitas <i>crawling</i>	Proses <i>crawling</i> dijalankan dari <i>executable</i> di disk dan menggunakan akses file/direktori untuk mengambil data, sehingga meninggalkan artefak.	Belum ada mekanisme untuk meminimalkan jejak <i>crawling</i> .	<i>Crawling</i> dilakukan dengan jejak minimal melalui <i>in-memory processing</i> , penghapusan <i>buffer</i> otomatis per <i>batch</i> , dan tanpa meninggalkan artefak yang mudah terdeteksi di disk.
Kanal eksfiltrasi dan karakteristik trafik	Eksfiltrasi menggunakan protokol umum tetapi pola trafik tidak disamarkan dan tetap mudah dibedakan dari komunikasi aplikasi sah.	Kanal komunikasi belum menyerupai trafik normal dan karakteristik paket belum tersamarkan sehingga aktivitas dapat terdeteksi.	Eksfiltrasi berlangsung melalui kanal yang menyerupai trafik sah dengan pola komunikasi adaptif, terenkripsi, dan tidak mencolok.
Mekanisme eksfiltrasi	Data hasil <i>crawling</i> langsung dikirim ke <i>endpoint</i> tanpa enkripsi atau kompresi khusus, sehingga struktur <i>payload</i> masih mudah diinterpretasi.	Tidak ada mekanisme untuk mengamankan <i>payload</i> eksfiltrasi.	Eksfiltrasi dilakukan dengan <i>payload</i> terenkripsi, dikemas, atau disamarkan sehingga tidak mudah dibaca maupun dianalisis.

III.2 Analisis Kebutuhan

III.2.1 Identifikasi Masalah Pengguna

Kelompok pengguna utama yang memanfaatkan hasil pengujian kapabilitas deteksi antivirus terhadap *spyware* mode *stealth* terdiri atas tiga kelompok: peneliti keamanan siber, pengembang *malware* uji, dan penyedia antivirus. Ketiganya berperan

penting dalam memastikan proses pengembangan, pengujian, serta analisis hasil deteksi berjalan aman, terkendali, dan memberikan hasil yang valid.

1. Peneliti Keamanan Siber

Kelompok ini memanfaatkan hasil pengujian untuk mengevaluasi efektivitas mekanisme deteksi terhadap ancaman *spyware* dengan mode *stealth* serta mengidentifikasi celah-celah deteksi yang masih ada. Temuan dari penelitian ini digunakan sebagai dasar pembuatan rekomendasi teknis dan strategi penguatan sistem pertahanan dengan tujuan meningkatkan kemampuan deteksi dan respons terhadap skenario pengumpulan data (*data crawling*) dan eksfiltrasi yang bersifat tersembunyi.

2. Pengembang *Malware* Uji

Kelompok ini bertugas membangun *spyware* dengan mode *stealth* yang meniru perilaku nyata, terutama pada tahap *data crawling* dan eksfiltrasi. Tantangan utama mereka adalah mengimplementasikan perilaku pengumpulan dan pengiriman data secara tersembunyi tanpa menyebabkan kerusakan nyata maupun terdeteksi terlalu dini oleh antivirus. Selain itu, pengembang harus memastikan keamanan lingkungan uji agar *malware* tidak keluar dari batas laboratorium.

3. Penyedia Antivirus atau EDR

Penyedia antivirus atau EDR menjadi pengguna yang memperoleh manfaat praktis dari hasil penelitian berupa bukti empiris terkait *blindspot* produk, rekomendasi konfigurasi telemetri, dan *scenario-driven test cases*. Informasi ini dapat digunakan untuk memperbaiki aturan deteksi, menambah cakupan pengumpulan telemetri, serta meningkatkan kebijakan respons insiden sehingga produk lebih tangguh terhadap varian *stealth* yang berkembang.

III.2.2 Kebutuhan Fungsional

Kebutuhan fungsional merupakan spesifikasi yang mendefinisikan fitur-fitur inti yang harus disediakan oleh *malware* uji agar dapat memenuhi tujuan dalam proses penelitian dan pengembangan. Kebutuhan ini dirancang untuk memastikan setiap kemampuan utama dapat dijalankan secara efektif dalam lingkungan uji sesuai skenario penelitian.

Tabel III.2 Kebutuhan Fungsional

ID	Kebutuhan	Penjelasan
FR-01	Sistem dapat melakukan proses <i>crawling</i> data secara <i>stealth</i>	Sistem harus memfasilitasi metode <i>crawling</i> bertahap (<i>low and slow</i>), memanfaatkan proses atau API sah (<i>living off the land</i>) agar pola akses menyerupai aktivitas normal sehingga sulit terdeteksi <i>anti-malware</i> .
FR-02	Sistem mendukung eksfiltrasi data menggunakan metode yang sulit dianalisis	Eksfiltrasi dilakukan melalui kanal terenkripsi (misal HTTPS, DNS <i>tunneling</i>), data dicicil secara acak, serta menyamarkan trafik sehingga tidak mudah dibedakan dari aktivitas jaringan normal oleh deteksi keamanan.
FR-03	Sistem tidak menyimpan artefak proses <i>crawling</i> atau eksfiltrasi di disk	Semua proses berjalan <i>in-memory</i> , tanpa membuat file <i>temporary</i> yang dapat dianalisis secara forensik, serta mendukung <i>memory wiping</i> setelah selesai eksekusi.
FR-04	Sistem memungkinkan penyembunyian aktivitas <i>crawling</i> dan eksfiltrasi dalam proses aplikasi sah	Aktivitas bisa disisipkan melalui teknik <i>injection/hollowing</i> ke aplikasi yang dipercaya, agar <i>event crawling</i> dan eksfiltrasi tersamarkan sebagai aktivitas aplikasi legal.
FR-05	Sistem dapat melakukan <i>clean up</i> otomatis untuk menghilangkan jejak digital	Setelah aktivitas <i>crawling</i> dan eksfiltrasi selesai, sistem menghapus artefak, jejak <i>log</i> , atau <i>payload</i> sementara sehingga <i>anti-malware</i> maupun forensik tidak menemukan indikasi aktivitas berbahaya.

III.2.3 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional mendefinisikan karakteristik kualitas, batasan, dan standar yang harus dipenuhi oleh *malware* uji selama pelaksanaan eksperimen. Kebutuhan ini berfokus pada aspek keamanan, kepatuhan, efisiensi, serta dampak perilaku *malware* di lingkungan pengujian untuk memastikan penelitian berjalan dengan aman, terkendali, dan sesuai prinsip etika akademik.

Tabel III.3 Kebutuhan Non-Fungsional

ID	Kebutuhan	Penjelasan
NFR-01	Kinerja	Sistem harus mampu melakukan <i>crawling</i> dan eksfiltrasi data tanpa menimbulkan konsumsi <i>resource</i> (CPU, memori, jaringan) yang tinggi agar tidak memicu alarm <i>anti malware</i> atau monitoring forensik.
NFR-02	Reliabilitas	Segala <i>failure</i> harus ditangani secara <i>silent</i> (tidak menimbulkan pesan <i>error/alert</i> pada <i>endpoint</i> atau sistem keamanan).
NFR-03	Keamanan lingkungan uji	Sistem harus memastikan seluruh eksekusi <i>malware</i> berlangsung dalam lingkungan <i>virtual/sandbox</i> yang benar-benar terisolasi dan tidak terhubung dengan sistem produksi atau jaringan publik.
NFR-04	Kepatuhan terhadap regulasi dan etika penelitian	Sistem wajib mematuhi regulasi keamanan siber, etika penelitian, serta tidak menggunakan <i>malware</i> untuk tujuan di luar pengujian akademik terisolasi.
NFR-05	Isolasi Dampak	Efek dan aktivitas <i>malware</i> harus benar-benar terbatas pada lingkungan uji, tidak boleh berdampak pada sistem luar (integritas lab terjaga).

III.3 Analisis Pemilihan Solusi

III.3.1 Alternatif Solusi

Permasalahan yang ditemukan dalam penelitian ini perlu dianalisis secara menyeluruh agar arah penyusunan solusi menjadi lebih terarah dan relevan dengan tujuan penelitian. Hasil identifikasi terhadap berbagai kendala yang muncul selama tahap studi literatur dan observasi awal kemudian dirangkum dalam Tabel ???. Tabel tersebut memuat daftar masalah utama beserta dampaknya terhadap proses *data crawling* dan eksfiltrasi dalam rangka menguji kapabilitas deteksi *anti-spyware* yang ada saat ini.

Tabel III.4 Masalah dan Dampak

ID	Masalah	Dampak
M-01	Proses <i>crawling</i> dijalankan dari <i>executable</i> di disk sehingga meninggalkan artefak.	Artefak di disk dapat dideteksi melalui <i>signature-based scanning</i> .
M-02	Pola akses sistem yang tidak sesuai <i>baseline</i> dapat terdeteksi sebagai aktivitas abnormal.	Aktivitas <i>crawling</i> atau pengumpulan data menjadi lebih mudah teridentifikasi oleh mekanisme deteksi berbasis perilaku, menurunkan tingkat <i>stealth</i> spyware.
M-03	Eksfiltrasi melalui protokol umum tanpa penyamaran pola trafik.	<i>Anti-spyware</i> dapat mendeteksi anomali trafik, sehingga proses eksfiltrasi berisiko terblokir atau memicu alarm keamanan.
M-04	Eksfiltrasi dengan volume data besar memunculkan pola trafik mencolok yang berbeda dari aktivitas normal.	Mekanisme deteksi dapat dengan mudah mengidentifikasi lonjakan <i>outbound traffic</i> sebagai anomali.

Pada tahap ini, beberapa solusi teknis diidentifikasi sebagai alternatif untuk menangani permasalahan proses *data crawling* dan eksfiltrasi dalam pengujian kapabilitas deteksi antivirus terhadap *spyware* dengan teknik *stealth* yang semakin berkembang. Setiap solusi menawarkan pendekatan berbeda dengan keunggulan dan keterbatasan masing-masing.

Tabel III.5 Analisis Alternatif Solusi

ID	Solusi	Kelebihan	Kekurangan
AS-01	<i>Living off the land</i>	Menggunakan <i>tools</i> OS yang trusted sehingga mirip aktivitas normal, mengurangi deteksi <i>signature</i> dan <i>behavior</i> .	Kegiatan yang terlalu mencurigakan masih bisa memicu alarm meskipun menggunakan <i>LOLBins</i> .
AS-02	Menyamarkan pola eksfiltrasi melalui pengiriman bertahap	Menghindari deteksi karena volume trafik terlihat seperti aktivitas normal.	Memperlambat proses eksfiltrasi sehingga total waktu pengiriman data menjadi jauh lebih lama.
AS-03	<i>Obfuscation custom payload</i> dan <i>command</i>	Mengubah <i>signature</i> dan pola sehingga mampu menghindari deteksi <i>signature-based</i> maupun <i>ML-based detection</i> .	Berisiko menghasilkan trafik/anomali mencurigakan jika obfuscation terlalu berat.
AS-04	<i>Anti-forensics</i> : hapus <i>memory artifacts</i> secara berkala	Mempersempit jejak forensik dan investigasi keamanan.	Menambah <i>overhead</i> dan berpotensi menghapus data yang masih diperlukan jika tidak diatur dengan tepat.
AS-05	<i>Fileless execution</i> + LOTL, <i>obfuscation</i> , dan eksfiltrasi <i>low profile</i>	Menurunkan jejak forensik dan membuat aktivitas spyware menyerupai proses sistem normal sehingga sangat sulit dideteksi oleh anti-spyware.	Lebih kompleks untuk diimplementasikan.

Alternatif solusi pertama (AS-01) menggunakan teknik *living off the land* di mana *spyware* memanfaatkan *tools* asli sistem operasi seperti PowerShell, WMI, dan *command line tools* yang sudah dipercaya sistem. Dengan menjalankan *crawling*

dan *eksfiltrasi* melalui *tools* bawaan ini, aktivitas *spyware* dilakukan secara “alami” dan menyerupai operasi sistem yang biasa, sehingga dapat mengurangi risiko deteksi oleh *anti malware* yang mengandalkan *signature* dan *behavior* pencarian program asing. Namun, keterbatasannya adalah fungsi *tool* bawaan yang tidak selalu cukup lengkap untuk semua kebutuhan *eksfiltrasi*, dan jika aktivitas terlalu intens, dapat memicu alarm *behavioral monitoring*.

Alternatif solusi kedua (AS-02) berfokus pada penyamaran pola *eksfiltrasi* melalui pengiriman data secara bertahap dalam ukuran kecil sehingga aliran trafik yang dihasilkan menyerupai aktivitas jaringan normal. Dengan memecah *payload* menjadi beberapa bagian dan mengatur interval pengiriman agar tidak membentuk lonjakan mencolok, proses *eksfiltrasi* tidak tampak seperti *transfer* data berukuran besar yang biasanya mudah dideteksi oleh sistem keamanan berbasis *anomaly detection*. Pendekatan ini efektif untuk menurunkan visibilitas *eksfiltrasi* di mata *anti-spyware* karena pola trafiknya mengikuti *baseline* komunikasi reguler pada host. Namun, konsekuensinya adalah waktu pengiriman data menjadi lebih lama dan *throughput* *eksfiltrasi* menurun, sehingga proses dapat terhambat apabila ukuran data besar atau koneksi tidak stabil.

Alternatif solusi AS-03 (*Obfuscation custom payload* dan *command*) melakukan teknik pengacakan dan pengenkripsian pada *payload malware*, perintah yang dikirim, serta komunikasi data *crawling* dan *eksfiltrasi*. Tujuannya adalah menghilangkan pola maupun *signature* yang mudah dikenali oleh *anti malware* dan sistem deteksi canggih berbasis *machine learning*. Dengan *obfuscation*, *malware* selalu tampak unik dan sulit dianalisis secara statis ataupun dinamis tanpa menimbulkan kecurigaan. Meski begitu, penggunaan *obfuscation* yang berlebihan dapat menimbulkan anomali trafik yang mudah dicurigai, dan dalam *sandbox* modern, teknik *obfuscation* semakin terbatas karena analisis dinamis kini lebih pintar dalam mendeteksi modifikasi kode maupun perilaku terselubung.

Alternatif solusi AS-04 menerapkan teknik *anti-forensics* dengan cara menghapus *memory artifacts* secara berkala untuk meminimalkan jejak yang dapat dianalisis oleh pihak keamanan atau investigator forensik. Pendekatan ini memastikan bahwa data sensitif, seperti hasil *crawling* sementara, *payload* sebelum dienkripsi, atau struktur *buffer*, tidak bertahan lama di memori. Dengan menghilangkan artefak secara rutin, *spyware* dapat mengurangi peluang terdeteksi oleh mekanisme keamanan yang memeriksa perubahan memori atau mencari pola residual dari aktivitas berbahaya. Namun, teknik ini memiliki risiko karena proses pembersihan yang terlalu

agresif dapat menghapus data yang masih dibutuhkan oleh modul *crawling* atau eksfiltrasi, serta menambah *overhead* eksekusi yang dapat mempengaruhi stabilitas operasi.

Alternatif solusi AS-05 menggabungkan teknik *fileless execution*, pemanfaatan *Living Off the Land* (LOTL) tools, *obfuscation*, serta eksfiltrasi *low profile* untuk meminimalkan jejak forensik dan meningkatkan kemampuan penyamaran aktivitas *spyware*. Dengan menjalankan instruksi langsung di memori dan memanfaatkan *executable* bawaan sistem, modul *spyware* dapat beroperasi tanpa meninggalkan file berbahaya di disk, sehingga perilakunya tampak seperti proses sistem normal di mata *anti-spyware*. *Obfuscation* pada perintah dan *payload* membantu menyembunyikan maksud sebenarnya dari analisis statis, sementara eksfiltrasi *low profile* memastikan transfer data berlangsung dalam pola kecil, teratur, dan menyerupai komunikasi reguler. Kombinasi teknik ini memberikan tingkat *stealth* yang tinggi, tetapi juga menambah kompleksitas implementasi karena membutuhkan kontrol eksekusi yang lebih halus dan penanganan yang tepat agar tidak menimbulkan *error* atau perilaku mencurigakan.

III.3.2 Analisis Penentuan Solusi

Analisis penentuan solusi dilakukan dengan pendekatan *Weighted Scoring Model* (WSM), yaitu metode yang memberi bobot serta skor pada setiap alternatif solusi berdasarkan sejumlah parameter penting yang telah ditetapkan. Setiap parameter seperti akurasi deteksi, efisiensi, skalabilitas, keamanan eksperimen, dan relevansi empiris diberi bobot sesuai tingkat kepentingannya dalam konteks pengujian *malware stealth* dan AV/EDR.

Selanjutnya, masing-masing solusi dievaluasi dan diberi skor pada rentang 1 (sangat buruk) sampai dengan 5 (sangat baik) untuk setiap parameter. Nilai total dari tiap alternatif dihitung dengan mengalikan skor pada setiap parameter dengan bobotnya, lalu menjumlahkan seluruh hasil tersebut. Rumus perhitungan dituliskan sebagai berikut:

$$\text{Skor Total Solusi} = \sum_i (\text{Skor Kriteria}_i \times \text{Bobot}_i)$$

Proses penilaian ini memberikan gambaran objektif mengenai keunggulan dan kelemahan relatif dari setiap opsi, sehingga solusi dengan nilai tertinggi secara ter-

ukur lebih unggul pada aspek-aspek utama penelitian. Dengan demikian, metode *weighted scoring* menjadi panduan ilmiah yang transparan dan sistematis dalam pemilihan pendekatan teknis, sekaligus menjaga hasil tetap relevan dan terfokus pada tujuan penelitian.

Tabel III.6 *Weighted Scoring Model*

Parameter	Bobot	AS-01	AS-02	AS-03	AS-04	AS-05
Kemampuan menghindari deteksi (<i>stealth</i>)	0.40	3	3	4	2	5
Kesesuaian dengan tujuan penelitian	0.30	3	2	3	2	4
<i>Overhead</i> kinerja	0.15	4	4	3	2	3
Risiko operasional/stabilitas	0.15	3	3	2	2	3
Total Skor		3.15	2.85	3.25	2.00	4.10

Berdasarkan hasil perhitungan *weighted scoring model*, alternatif solusi AS-05, yaitu *fileless execution* dengan kombinasi LOLTL, *obfuscation*, dan eksfiltrasi *low profile*, dipilih sebagai opsi terbaik. Solusi ini memperoleh skor total tertinggi karena menunjukkan performa optimal hampir di seluruh parameter utama, khususnya pada aspek akurasi *stealth*, kesesuaian dengan tujuan penelitian, *overhead* kinerja, serta risiko operasional dan stabilitas.

Pendekatan *fileless execution* memungkinkan *spyware* berjalan sepenuhnya di memori, sehingga meminimalkan artefak pada disk dan menurunkan kemungkinan terdeteksi melalui pemindaian berbasis *signature* maupun analisis forensik. Dengan memanfaatkan LOLBins/LOLBAS (tool bawaan OS yang dipercaya), proses *crawling* dan eksfiltrasi data dapat dilakukan tanpa meninggalkan jejak mencolok, sementara teknik *obfuscation* serta kanal eksfiltrasi *low profile* semakin memperkuat kemampuan penyamaran aktivitas *spyware*.

Meskipun implementasinya lebih kompleks, AS-05 memberikan keleluasaan kontrol bagi *engineer* untuk mengatur berbagai skenario *evasive* dan menguji ketahanan sistem AV/EDR secara menyeluruh. Selain itu, solusi ini tetap memperhatikan kestabilan operasional selama eksperimen melalui perancangan lingkungan yang terisolasi serta penerapan mekanisme pemantauan risiko yang disiplin, sehingga keamanan lingkungan pengujian tetap terjaga.

BAB IV

DESAIN KONSEP SOLUSI

Bab ini berisi penjelasan secara rinci mengenai rancangan sistem yang diusulkan dalam penelitian ini, yaitu desain modul *mission* dan *exfiltration* pada *spyware* dengan mode *stealth* yang digunakan untuk menguji kapabilitas deteksi *anti-spyware* dalam lingkungan terisolasi.

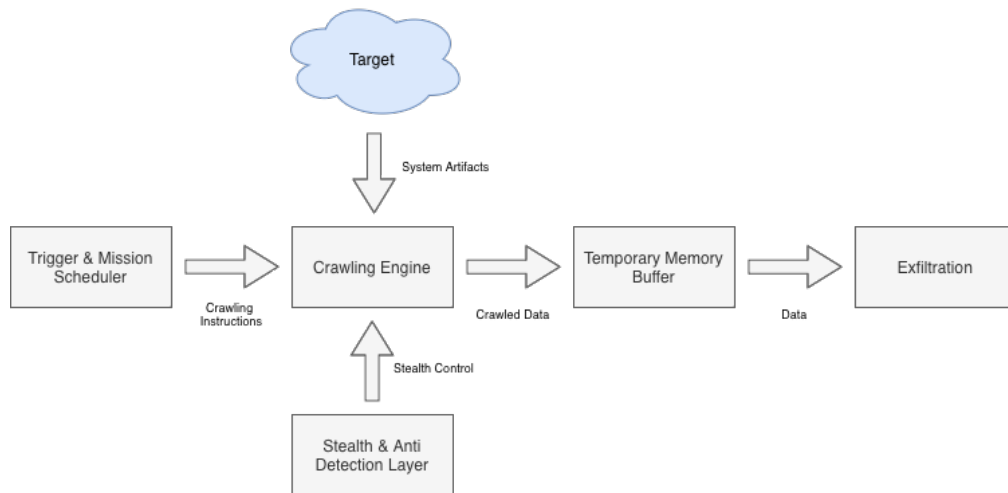
IV.1 Tahapan Desain

IV.1.1 Rancangan Modul *Mission*

Rancangan modul *Mission* disusun untuk mendefinisikan bagaimana proses pengambilan data (*data crawling*) dijalankan secara tersembunyi (*stealth*) pada sistem target. Modul ini berfungsi sebagai komponen utama yang menangani identifikasi artefak target, eksekusi proses *crawling* secara terjadwal, serta pengendalian tingkat aktivitas agar tidak menimbulkan pola perilaku mencurigakan. Seluruh mekanisme di dalamnya dirancang dengan pendekatan *fileless execution*, pengacakan interval, dan lapisan *anti detection* untuk memastikan bahwa proses berjalan tanpa memicu peringatan dari mekanisme keamanan. Bagian ini mencakup arsitektur modul, jenis data yang menjadi target *crawling*, lingkungan operasi, serta teknik-teknik *crawling stealth* yang digunakan dalam pengembangan sistem.

IV.1.1.1 Arsitektur Modul *Mission*

Modul *mission* berfungsi sebagai inti proses *crawling* data dalam sistem *spyware* mode *stealth*, dengan tujuan utama melakukan pengumpulan informasi secara diam-diam tanpa terdeteksi perangkat keamanan. Arsitektur modul ini mengutamakan teknik *fileless* dan *evasion* agar proses kerja tidak meninggalkan artefak digital di disk, serta meminimalisir kemungkinan identifikasi oleh *anti spyware*.



Gambar IV.1 Diagram Blok Arsitektur Modul *Mission*

Secara umum, arsitektur modul *Mission* yang akan dirancang terdiri dari empat komponen utama, yaitu:

1. *Trigger & Mission Scheduler*

Trigger & mission scheduler merupakan lapisan yang bertanggungjawab memulai eksekusi misi berdasarkan interval waktu, kondisi sistem, atau pemicu tertentu. *Scheduler* memastikan bahwa proses *crawling* tidak berjalan secara terus-menerus, melainkan dieksekusi dalam *batch* kecil dengan pola waktu yang acak. Pendekatan ini bertujuan untuk menghindari karakteristik eksekusi berulang dan berurutan, karena pola tersebut sering dianggap mencurigakan oleh sistem deteksi berbasis perilaku.

2. *Data Crawling Engine*

Komponen *crawling engine* merupakan inti dari proses pengambilan data pada sistem target. *Engine* ini menangani tugas enumerasi artefak, pembacaan *file* atau metadata, serta pengambilan informasi sensitif lainnya yang relevan. Proses *crawling* dirancang agar bersifat modular sehingga dapat menyesuaikan jenis artefak yang dituju. Seluruh proses berjalan dengan intensitas yang dikendalikan, menggunakan API bawaan sistem dan teknik khusus seperti *low-frequency access*, sehingga aktivitas *crawling* menyerupai perilaku rutin sistem operasi. Dengan begitu, *crawling engine* dapat mengakses data yang diperlukan tanpa menimbulkan lonjakan I/O atau pola akses abnormal yang dapat memicu alarm *anti spyware*.

3. *Stealth & Anti Detection Layer*

Komponen *stealth & anti detection layer* berfungsi untuk menjaga agar seluruh aktivitas modul *Mission* tetap tersembunyi dari mekanisme *anti spyware*.

Pada lapisan ini diimplementasikan *anti VM*, *anti sandbox*, dan *anti debugging* yang memungkinkan modul mendeteksi lingkungan yang tidak aman dan menunda proses sampai kondisi normal. Selain itu, lapisan ini mengontrol batasan intensitas *crawling* melalui pengaturan waktu jeda, pembatasan durasi eksekusi, serta pengacakan interval. Teknik *living-off-the-land* (LOTL) seperti pemanfaatan PowerShell, WMI, atau proses-proses internal sistem juga digunakan untuk menyamarkan jejak eksekusi.

4. *Stealth & Anti Detection Layer*

Temporary memory buffer merupakan ruang penyimpanan sementara yang menampung hasil *crawling* sebelum proses eksfiltrasi dilakukan. Sesuai dengan prinsip *memory-resident* pada *fileless architecture*, *buffer* ini diletakkan sepenuhnya di memori untuk menghindari pembuatan artefak fisik pada disk yang dapat menjadi bukti forensik. Data yang terkumpul disimpan dalam bentuk terstruktur dan dapat mengalami proses kompresi ringan agar lebih efisien. *Buffer* juga dilengkapi mekanisme penghapusan otomatis setelah sesi eksfiltrasi selesai, sehingga tidak meninggalkan residu yang dapat dianalisis. *Buffer* juga menjadi indikator kesiapan eksfiltrasi karena proses pengiriman data hanya dilakukan ketika kapasitas tertentu tercapai atau ketika misi menyelesaikan satu *batch crawling*.

Untuk mewujudkan arsitektur modul *Mission*, diperlukan pemilihan teknologi yang tepat agar setiap komponen dapat berfungsi sesuai karakteristiknya. Arsitektur tersebut memerlukan mekanisme eksekusi yang ringan, fleksibel, serta mampu berjalan sepenuhnya secara *fileless* tanpa meninggalkan jejak digital pada *disk*. Selain itu, modul *Mission* harus mendukung teknik *living-off-the-land*, implementasi *anti analysis*, serta pengelolaan data *in memory* agar aktivitas *crawling* dapat berlangsung secara *stealth* dan tidak memicu sistem deteksi *anti spyware*. Oleh karena itu, teknologi yang digunakan harus mampu mengakomodasi operasi *low level* pada sistem, menyediakan akses ke artefak sistem, serta sekaligus mendukung eksekusi yang tersamarkan. Pada Tabel IV.1 dibahas teknologi yang digunakan untuk mengimplementasikan setiap komponen modul *Mission* serta alasan pemilihannya.

Tabel IV.1 Teknologi yang Digunakan pada Setiap Komponen Modul *Mission*

Komponen Modul Mission	Teknologi yang Digunakan
Trigger & Mission Scheduler	• Python (time, random, asyncio)
Data Crawling Engine	• Python (os, pathlib, psutil, subprocess) • PowerShell • WMI • Win32 API
Stealth & Anti Detection Layer	• Win32 API • PowerShell • LOLBins
Temporary Memory Buffer	• Python (io.BytesIO, zlib)

IV.1.1.2 Target Data dan Lingkungan Operasi

Modul *Mission* dirancang untuk mengumpulkan data yang relevan dengan pemetaan aktivitas sistem dan pengguna, serta mendukung proses analitik pada tahap selanjutnya. Data yang dikumpulkan berfokus pada artefak yang dapat memberikan gambaran mengenai perilaku sistem tanpa mengambil konten sensitif secara langsung. Secara garis besar, terdapat X kelompok utama data yang menjadi target *crawling*.

Tabel IV.2 Target Data

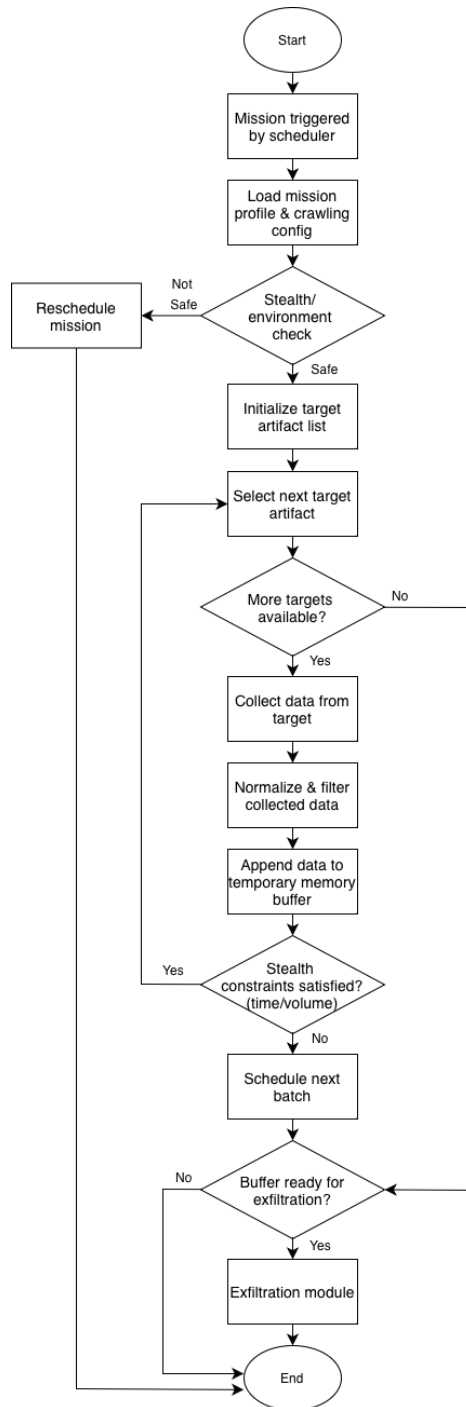
Kategori Data	Deskripsi	Atribut Utama	Contoh Nilai
Hasil <i>Keylogging</i>	Rekaman aktivitas penekanan tombol secara kronologis.	timestamp, key_code, process_name	–
Metadata Proses Sistem	Informasi konteks eksekusi proses pada saat aktivitas terjadi.	pid, process_name, cpu_usage, memory_usage, start_time, end_time	–
Artefak Sistem Tambahan	Artefak yang menggambarkan kondisi sistem saat <i>crawling</i> .	active_window, running_processes, focus_change	–

Seluruh proses *crawling* pada modul *Mission* dijalankan di dalam lingkungan terisolasi menggunakan *Virtual Machine (VM)* berbasis Windows 10/11 (64-bit). Penggunaan *VM* dipilih untuk memastikan bahwa pengujian berlangsung secara aman, tidak

memengaruhi sistem utama, serta memungkinkan proses *rollback* melalui *snapshot* apabila terjadi kesalahan selama eksperimen. Modul berjalan pada level pengguna (*user-level access*) tanpa hak administratif, sehingga mencerminkan kondisi bagaimana *spyware* berbasis *stealth* beroperasi. Selain itu, proses *crawling* hanya dilakukan apabila kondisi lingkungan dinilai aman untuk menjaga karakteristik *stealth* dari modul yang dirancang.

IV.1.1.3 Alur Kerja Modul *Mission*

Alur kerja modul *Mission* menggambarkan urutan proses yang dijalankan selama tahap pengumpulan data berlangsung, mulai dari pemicu eksekusi, pemeriksaan kondisi lingkungan, hingga penyimpanan data sementara dan persiapan eksfiltrasi. Secara keseluruhan, *workflow* dirancang agar proses *crawling* berjalan secara adaptif, terkontrol, dan tetap mempertahankan karakteristik *stealth* agar tidak terdeteksi oleh mekanisme pertahanan sistem. Diagram alur pada Gambar IV.2 memperlihatkan langkah-langkah utama yang dilakukan oleh modul ini.



Gambar IV.2 *Flowchart* Alur Kerja Modul *Mission*

Proses dimulai ketika *Mission Scheduler* memicu eksekusi modul sesuai interval waktu yang telah diacak untuk menghindari pola statis. Setelah modul mulai berjalan, konfigurasi misi dan profil *crawling* dimuat ke dalam memori untuk menentukan jenis data yang harus dikumpulkan pada *batch* eksekusi tersebut. Sebelum proses *crawling* dilakukan, modul menjalankan *stealth/environment check* untuk memas-

tikan bahwa kondisi perangkat aman. Pada tahap ini, sistem memeriksa indikasi keberadaan *sandbox*, *debugger*, atau aktivitas sistem yang tidak wajar. Jika lingkungan dianggap tidak aman, misi tidak dilanjutkan dan dijadwalkan ulang pada waktu berikutnya.

Apabila kondisi aman, modul *Mission* menginisialisasi daftar artefak target yang akan diproses, seperti metadata proses aktif atau *buffer keylogging* yang perlu dipindai. Modul kemudian memilih target berikutnya dari daftar, dan memeriksa apakah masih terdapat artefak yang harus dikumpulkan. Jika tidak ada target tersisa, *batch crawling* dianggap selesai dan proses dapat berlanjut ke tahap berikutnya.

Untuk setiap target yang tersedia, modul mengambil data melalui *crawling engine*. Proses ini mencakup pembacaan metadata sistem maupun pengambilan *buffer* hasil penekanan tombol. Data mentah yang diperoleh kemudian dinormalisasi, difilter, dan disesuaikan dengan format penyimpanan sementara agar tetap efisien dan konsisten. Setelah diproses, data ditambahkan ke dalam *temporary memory buffer* yang berada sepenuhnya di memori untuk menjaga sifat *fileless*.

Selanjutnya, modul melakukan evaluasi terhadap batasan *stealth*, seperti durasi eksekusi maksimum atau volume data per *batch*. Jika batasan belum terpenuhi, modul menjadwalkan *batch crawling* berikutnya dan kembali ke proses pemilihan target. Jika batasan terpenuhi, modul bergerak ke tahap selanjutnya, yaitu memeriksa apakah *buffer* siap untuk dieksfiltrasi. Pemeriksaan ini dapat berbasis ukuran *buffer*, jumlah entri yang terkumpul, atau kondisi khusus lainnya.

Jika *buffer* belum memenuhi kriteria eksfiltrasi, modul kembali ke proses *crawling* pada *batch* berikutnya. Namun, apabila *buffer* telah mencapai kondisi siap kirim, modul *Mission* mengaktifkan komponen eksfiltrasi dan menyerahkan data untuk dikirim ke modul berikutnya. Setelah proses eksfiltrasi selesai, eksekusi modul *Mission* pada *batch* tersebut diakhiri.

Dengan alur kerja ini, modul *Mission* dapat melakukan pengumpulan data secara bertahap, efisien, dan tetap mempertahankan sifat *stealth*. Pendekatan berbasis *batch*, validasi lingkungan, dan kontrol frekuensi eksekusi memastikan proses berjalan tanpa memicu sistem deteksi maupun meninggalkan artefak yang dapat dianalisis.

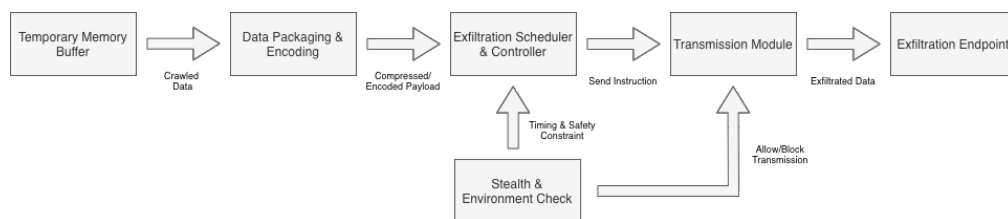
IV.1.2 Rancangan Modul *Exfiltration*

Rancangan modul *Exfiltration* disusun untuk mendefinisikan bagaimana data hasil *crawling* diproses, dikemas, dan dikirimkan ke *endpoint* secara aman serta tetap

mempertahankan karakteristik *stealth*. Modul ini berfungsi mengelola penjadwalan pengiriman, menyiapkan *payload* yang telah terstruktur, serta memastikan proses transmisi berlangsung tanpa memicu deteksi dari sistem keamanan. Mekanisme di dalamnya mencakup proses *packaging*, *encoding*, *compression*, pemilihan waktu dan metode pengiriman, serta kontrol lingkungan sebelum data dikirim. Bagian ini membahas arsitektur modul eksfiltrasi, jenis *output* yang dihasilkan, serta alur mekanisme pengiriman data dari *buffer* hingga diterima oleh *endpoint* analitik.

IV.1.2.1 Arsitektur Modul *Exfiltration*

Modul *Exfiltration* bertanggungjawab untuk mengirimkan data hasil *crawling* dari modul *Mission* ke penerima dalam bentuk yang terstruktur, aman, dan tetap mempertahankan sifat *stealth*. Modul *Exfiltration* berperan sebagai jembatan antara lingkungan eksekusi *spyware* dan *big data analytic*. Arsitektur modul ini dirancang agar dapat bekerja secara *fileless*, meminimalkan jejak pada *disk*, serta menggunakan pola pengiriman *low and slow* sehingga tidak memicu deteksi anomali jaringan.



Gambar IV.3 Diagram Blok Arsitektur Modul *Exfiltration*

Secara garis besar, arsitektur modul *Exfiltration* terdiri dari beberapa komponen utama, yaitu:

1. *Temporary Memory Buffer*

Temporary memory buffer berfungsi sebagai titik masuk (*entry point*) bagi modul *Exfiltration*. *Buffer* ini berisi kumpulan data hasil *crawling* modul *Mission* yang disimpan sepenuhnya di memori. Dengan menempatkan *buffer* di RAM, arsitektur ini menghindari pembuatan *file log* permanen pada *disk* sehingga konsisten dengan pendekatan *fileless*. Modul *Exfiltration* tidak berinteraksi langsung dengan modul *Mission*, melainkan hanya membaca data yang sudah siap dari *buffer* tersebut.

2. *Data Packaging & Encoding*

Data packaging & encoding bertugas mengubah data mentah dalam *buffer* menjadi *payload* yang siap dikirim. Pada tahap ini dilakukan proses normalisasi, penggabungan, kompresi, serta encoding data. Tujuannya adalah meng-

urangi ukuran *payload*, menyamakan format antar *batch*, dan mempermudah proses pengiriman melalui protokol jaringan yang digunakan. Dengan demikian, data yang dikirim ke *endpoint* bersifat ringkas, konsisten, dan tidak mudah dikenali sebagai *traffic* yang tidak biasa.

3. *Exfiltration Scheduler & Controller*

Komponen *exfiltration scheduler & controller* berperan sebagai pengatur waktu dan ritme pengiriman. Modul ini menentukan kapan proses eksfiltrasi dijalankan, seberapa besar ukuran *batch* yang dikirim, serta mengatur interval pengiriman agar tidak menimbulkan lonjakan lalu lintas jaringan. Pendekatan *low frequency transmission* dan *randomized interval* diterapkan juga diterapkan sehingga pola pengiriman tidak tampak statis atau berulang. *Scheduler* juga mengelola status antrian pengiriman, misalnya menandai *payload* yang telah berhasil dikirim maupun yang perlu diulang.

4. *Stealth & Environment Check*

Komponen *stealth & environment check* yang memonitor kondisi lingkungan sebelum dan selama proses eksfiltrasi. Komponen ini melakukan pengecekan terhadap indikator tertentu, seperti beban CPU, ketersediaan jaringan, maupun adanya tanda-tanda lingkungan analisis (*sandbox*, *debugger*, atau *monitoring* intensif). Hasil pengecekan ini digunakan sebagai *constraint* bagi *exfiltration scheduler & controller* dan *transmission module*. Jika kondisi tidak memenuhi kriteria *stealth*, modul dapat menunda pengiriman, memperpanjang interval, atau membatalkan proses eksfiltrasi untuk *batch* tersebut.

5. *Transmission Module*

Komponen *transmission module* merupakan bagian yang melakukan pengiriman data secara nyata ke pihak penerima. Modul ini mengolah *payload* yang sudah dikemas dan apabila sudah diizinkan oleh *stealth layer* mengirimkannya melalui saluran komunikasi yang telah ditentukan. *Transmission module* didesain agar menyerupai trafik aplikasi biasa, sehingga tidak mudah dibedakan dari aktivitas jaringan normal oleh mekanisme pemantauan.

6. *Exfiltration Endpoint*

Exfiltration endpoint merupakan komponen di sisi penerima yang menerima *payload* dari modul *exfiltration*. *Endpoint* ini berupa server penerima di lingkungan terkontrol yang kemudian akan meneruskan data tersebut ke sistem penyimpanan yang akan diolah dalam *big data analytic*.

IV.1.2.2 Target Output *Exfiltration*

Output eksfiltrasi merupakan data akhir yang telah dikumpulkan oleh modul *Mission* dan diproses sebelum dikirimkan ke *endpoint* tujuan. Secara umum, output yang dieksfiltrasi terdiri dari tiga kategori utama yang disajikan pada Tabel IV.3.

Tabel IV.3 Target *Output Exfiltration*

Kategori Output	Deskripsi	Atribut Utama	Contoh Nilai
Hasil <i>Keylogging</i>	Rekaman aktivitas penekanan tombol secara kronologis.	timestamp, key_code, process_name	–
Metadata Proses Sistem	Informasi konteks eksekusi proses pada saat aktivitas terjadi.	pid, process_name, cpu_usage, memory_usage, start_time, end_time	–
Artefak Sistem Tambahan	Artefak yang menggambarkan kondisi sistem saat <i>crawling</i> .	active_window, running_processes, focus_change	–

Sebelum dikirimkan, seluruh data dikemas dalam struktur *payload* yang sudah dikompresi dan di-*encode*. *Payload* dapat berupa JSON terstruktur sebelum diberikan proses kompresi. Struktur *payload* dipaparkan pada Tabel IV.4.

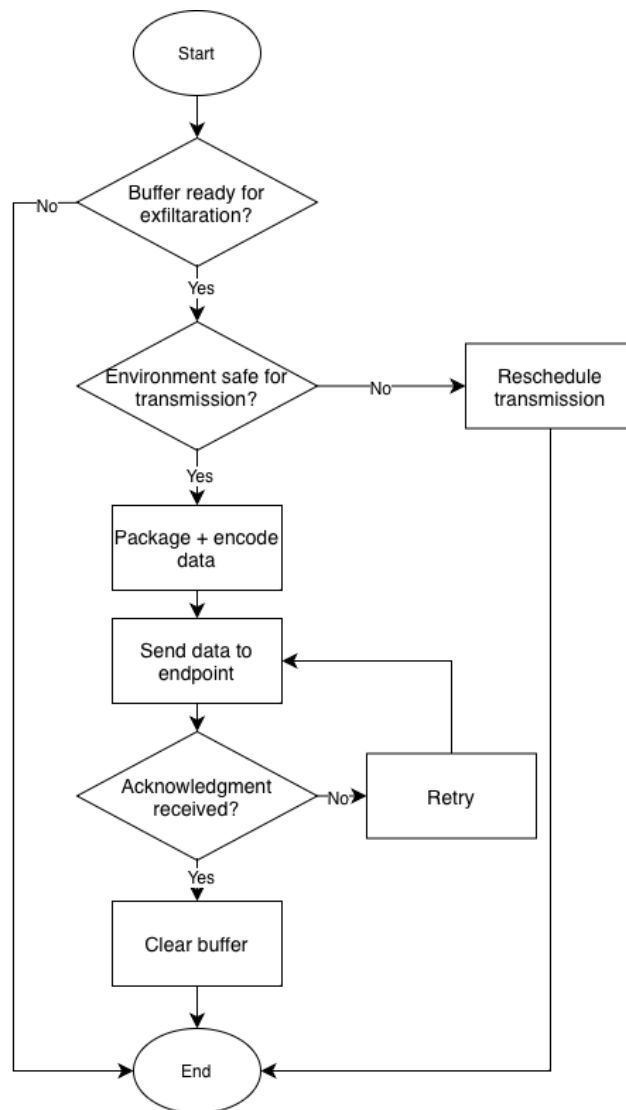
Tabel IV.4 Struktur *Payload*

<i>Field</i>	Isi/Struktur
keylogs	List objek <i>keylogging</i>
metadata	List metadata proses
system_artifacts	Data artefak sistem
session_id	ID unik <i>batch crawling</i>
generated_at	<i>Timestamp</i> waktu <i>payload</i> dibuat

IV.1.2.3 Mekanisme Pengiriman Data

Mekanisme pengiriman data pada modul *Exfiltration* dirancang untuk memastikan proses transfer berjalan secara aman, terkontrol, dan tidak menimbulkan pola komunikasi yang mudah dikenali oleh sistem pertahanan. Alur kerja ini digambarkan melalui diagram alur pada Gambar IV.4, dimulai dari pemeriksaan kesiapan *buffer*,

penjadwalan waktu pengiriman, hingga proses pembersihan *buffer* setelah data berhasil dikirimkan.



Gambar IV.4 *Flowchart* Mekanisme Pengiriman Data pada Modul *Exfiltration*

Proses dimulai dengan memeriksa apakah *temporary memory buffer* telah mencapai kondisi siap untuk dikirim, baik berdasarkan ukuran muatan (*payload size threshold*) maupun batas waktu (*time threshold*). Jika *buffer* belum siap, *batch* dihentikan dan modul menunggu siklus berikutnya yang dipicu kembali oleh *scheduler*.

Apabila *buffer* berada dalam kondisi siap, modul akan melakukan evaluasi lingkungan. Tahap ini memastikan bahwa sistem target berada dalam keadaan aman untuk melakukan komunikasi keluar. Jika kondisi dinilai tidak aman, modul tidak memaksakan pengiriman dan akan menjadwalkan ulang transmisi pada interval ber-

ikutnya.

Jika lingkungan aman, sistem memasuki tahap *data packaging & encoding*. Pada tahap ini, data hasil *crawling* yang tersimpan di memori dikompresi dan di-*encode* menggunakan teknik ringan agar ukuran paket lebih kecil dan pola data lebih sulit dianalisis. *Payload* yang sudah dipaketkan kemudian dikirimkan ke *endpoint* tujuan melalui modul transmisi.

Setelah pengiriman dilakukan, modul akan menunggu respons berupa *acknowledgment* dari *endpoint*. Jika *acknowledgment* tidak diterima, modul akan mencoba melakukan *retry* pengiriman sejumlah batas maksimum yang telah ditentukan. Proses *retry* tetap berada dalam *batch* yang sama. Jika seluruh percobaan gagal, modul akan mengakhiri *batch* tanpa menghapus *buffer* sehingga data tetap aman dan akan dicoba kembali pada siklus berikutnya.

Apabila *acknowledgment* diterima, *buffer* dianggap telah berhasil dieksfiltrasi. Modul kemudian melakukan *buffer clearing* untuk menghapus seluruh data yang telah dikirim dari memori, sesuai dengan prinsip *memory resident operation* pada arsitektur *fileless*. Tahap ini sekaligus menandai berakhirnya satu siklus eksfiltrasi, dan modul kembali menunggu pemicu berikutnya dari *scheduler*.

IV.1.2.4 Rancangan Lingkungan Pengujian

Lingkungan pengujian dirancang agar proses eksekusi modul *Mission* dan *Exfiltration* dapat dilakukan secara aman dan terkontrol. Seluruh percobaan dilakukan pada lingkungan Windows yang dikonfigurasi secara terisolasi sehingga aktivitas *crawling* dan eksfiltrasi tidak berdampak pada sistem di luar area pengujian. Lingkungan uji yang digunakan disajikan pada Tabel IV.5.

Tabel IV.5 Konfigurasi Lingkungan Pengujian

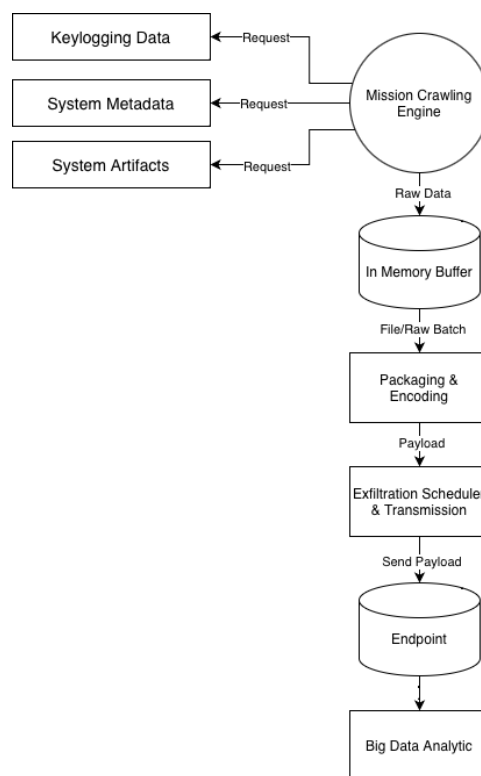
Komponen	Detail Konfigurasi
Sistem Operasi	Windows 10/11
Bahasa Pemrograman	Python 3.x, PowerShell
Library Utama	psutil, pathlib, requests, Win32API
Endpoint Eksfiltrasi	REST API (FastAPI)

Lingkungan pengujian pada penelitian ini dibangun menggunakan sistem operasi Windows 10/11 yang dikonfigurasi secara terisolasi untuk memastikan bahwa pro-

ses *crawling* dan *eksfiltrasi* tidak berdampak pada sistem di luar area eksperimen. Seluruh modul dijalankan menggunakan Python 3.x dan PowerShell sebagai bahasa pemrograman utama, dengan dukungan beberapa *library* seperti *psutil*, *pathlib*, *requests*, dan Win32API untuk mengakses artefak sistem serta mengelola proses *crawling*. Proses *exfiltration* diarahkan menuju sebuah *endpoint* lokal berbasis *REST API* (FastAPI) yang berfungsi sebagai server penerima *payload* hasil *crawling*. Konfigurasi ini memastikan bahwa seluruh aktivitas uji dapat dilakukan secara aman, terkontrol, dan sesuai dengan prinsip penelitian *spyware mode stealth*.

IV.2 Hasil Desain

Bagian ini merupakan hasil desain dari arsitektur sistem yang mencakup keseluruhan proses *mission crawling* hingga *data exfiltration*, yang menjadi fokus utama pada penelitian ini. Desain ini disusun untuk menunjukkan bagaimana modul-modul yang dibangun berinteraksi, bagaimana data bergerak dari sumber hingga tujuan akhir, serta bagaimana prinsip *stealth* dipertahankan sepanjang alur eksekusi.



Gambar IV.5 Diagram Hasil Desain Arsitektur Sistem

Pada desain ini, sistem memulai proses dengan tiga kelompok data utama, yaitu *keylogging data*, *system metadata*, dan *system artifacts*. Ketiganya diakses oleh *Mis-*

sion Crawling Engine secara terjadwal melalui mekanisme *request based crawling*. *Engine* kemudian menghasilkan *raw data* yang disimpan di *In Memory Buffer*, yaitu ruang penyimpanan sementara yang bersifat *memory resident* untuk menghindari terbentuknya jejak digital pada *disk*.

Selanjutnya, data dari *buffer* diproses dalam tahap *Packaging & Encoding*, di mana data dikemas, di-*encode*, dan dikompresi menjadi sebuah *payload* terstruktur. *Payload* ini kemudian dikirim melalui *Exfiltration Scheduler & Transmission Module*, yang bekerja dengan memperhatikan batasan *stealth* seperti pengacakan waktu pengiriman, pemilihan metode transmisi yang rendah anomali, serta mekanisme *retry* bila konfirmasi tidak diterima. Data yang berhasil dieksfiltrasi akan tersimpan dalam *Endpoint*, dan selanjutnya menjadi input bagi *Big Data Analytic* untuk proses analisis lebih lanjut.

BAB V

RENCANA SELANJUTNYA

Bab ini berisi rencana implementasi yang akan dilakukan untuk merealisasikan desain yang telah dirumuskan pada Desain Solusi. Bab ini menyajikan rencana selanjutnya yang mencakup tiga bagian utama, yaitu rencana implementasi prototipe, rencana evaluasi terhadap hasil pengujian, serta analisis risiko yang mungkin muncul selama proses tersebut.

V.1 Rencana Implementasi

V.1.1 Rencana Implementasi Modul *Mission*

Implementasi modul *mission* dilakukan dalam lingkungan uji terisolasi untuk memastikan seluruh aktivitas data *crawling* dapat diamati tanpa risiko menyebarnya malware uji ke lingkungan luar. Lingkungan terisolasi ini memungkinkan proses pengamatan berjalan aman tanpa mengganggu sistem utama, serta memberikan ruang untuk melakukan *rollback* apabila terjadi gangguan selama pengujian. Rencana implementasi modul *mission* disajikan pada Tabel V.1.

Tabel V.1 Rencana Implementasi Modul *Mission*

Langkah	Penjelasan	Tools
Penentuan target data <i>crawling</i>	Mengidentifikasi jenis data yang akan diambil, yaitu hasil <i>keylogging</i> , metadata proses sistem, artefak sistem tambahan, dan aktivitas <i>cache</i> internet <i>browser</i> .	Data <i>dummy</i>
Implementasi <i>file-less execution</i>	Menjalankan modul <i>mission</i> sepenuhnya di memori dengan <i>PowerShell/WMI</i> agar tidak meninggalkan artefak <i>disk</i> .	<i>PowerShell</i> , <i>WMI</i>
Pemanfaatan <i>LOL-Bins</i> untuk proses <i>crawling</i>	Memanfaatkan perintah bawaan Windows dan query <i>WMI</i> untuk membaca direktori, <i>registry</i> ringan, serta metadata sistem. Pendekatan ini menyamarkan <i>crawling</i> sebagai aktivitas legal.	Windows native tools
Pengaturan pola <i>crawling low and slow</i>	Mengatur intensitas <i>crawling</i> agar berjalan perlahan dan acak, dengan interval yang tidak konstan serta pembacaan <i>directory</i> per- <i>batch</i> kecil. Tujuannya meniru pola pengguna normal dan memperkecil risiko <i>alert</i> dari sistem keamanan.	Scheduler
Pembentukan <i>buffer</i> data in memory	Data hasil <i>crawling</i> disimpan sementara ke dalam <i>buffer</i> di memori sebelum dikirimkan ke modul eksfiltrasi. <i>Buffer</i> dibersihkan secara berkala untuk menghindari jejak forensik.	Memory buffer module
<i>Obfuscation</i> ringan pada instruksi <i>crawling</i>	<i>Payload</i> dan instruksi <i>crawling</i> diberi <i>encoding</i> atau variasi nama parameter untuk mengurangi deteksi <i>signature</i> , tanpa menimbulkan pola anomali baru.	<i>Encoder</i>
Verifikasi fungsi modul <i>mission</i>	Menguji apakah <i>crawling</i> berjalan stabil, tidak memicu <i>alert</i> awal, dan <i>buffer</i> terbentuk dengan benar dalam lingkungan terisolasi. Dilakukan sebelum integrasi dengan modul eksfiltrasi.	Host Windows 10/11

V.1.2 Rencana Implementasi Modul *Exfiltration*

Modul *exfiltration* bertugas mengirimkan data hasil proses *mission* ke server penerima. Rencana implementasi disusun untuk memastikan proses pengiriman data berjalan terstruktur, aman, dan konsisten. Rencana implementasi modul *exfiltration* disajikan pada Tabel V.2.

Tabel V.2 Rencana Implementasi Modul *Exfiltration*

Langkah	Penjelasan	Tools
Menentukan se- rver penerima	Menyusun layanan sederhana pada sisi penerima untuk menerima data hasil eksfiltrasi dari host Windows.	Python <i>FastAPI</i>
Mempersiapkan struktur data yang akan dikirim	Data hasil <i>crawling</i> yang sudah disimpan sementara dalam <i>buffer in memory</i> dikemas menjadi format JSON agar mudah dikirim dan tidak menimbulkan <i>burst traffic</i> mencolok.	<i>Modul</i> formatting JSON
Pengaturan interval pengiriman acak (<i>low and slow</i>)	Setiap <i>payload</i> dikirim dalam interval waktu acak dan tidak beraturan sehingga pola komunikasi tidak menunjukkan ritme repetitif yang dapat memicu deteksi anomali.	Timer/Scheduler
Mengirimkan data melalui kanal komunikasi umum (HTTP/S)	Eksfiltrasi dilakukan menggunakan <i>request</i> HTTP/HTTPS lokal sehingga pola komunikasinya menyerupai aplikasi biasa.	PowerShell
Pembersihan <i>buffer</i> setelah pengiriman	Setelah setiap potongan data dikirim, <i>buffer</i> dihapus dari memori untuk meningkatkan <i>stealth</i> dan mengurangi jejak aktivitas pada sistem.	Memory buffer module
Verifikasi pengiriman data	Memastikan setiap data berhasil diterima oleh server lokal tanpa <i>error</i> .	Log server

V.2 Rencana Evaluasi

V.2.1 Parameter dan Metrik Evaluasi

Parameter dan metrik evaluasi digunakan untuk menentukan aspek apa saja yang perlu diamati selama pengujian serta bagaimana kualitas perilaku modul diukur. Penetapan parameter membantu memastikan setiap aktivitas *mission* dan *exfiltration* dapat dievaluasi secara objektif, sementara metrik yang ditentukan berfungsi sebagai indikator keberhasilan atau kegagalan pada setiap skenario uji.

Tabel V.3 Parameter dan Metrik Evaluasi

Parameter Evaluasi	Penjelasan	Metrik yang Diukur
Deteksi pada tingkat proses	Mengamati apakah aktivitas modul <i>mission</i> terdeteksi oleh produk keamanan.	• Jumlah <i>alert</i> • Jenis <i>alert</i> (<i>warning/block</i>) • Kategori deteksi (<i>behavioral/signature based</i>)
Deteksi aktivitas <i>in-memory/fileless</i>	Mengevaluasi apakah eksekusi di memori (tanpa menulis <i>file</i>) teridentifikasi oleh sistem.	• <i>Event log</i> ID yang muncul • <i>Flag</i> atau indikator dari <i>anti-spyware</i> • Anomali proses <i>in-memory</i>
Deteksi pada aktivitas <i>crawling</i>	Melihat apakah enumerasi direktori, pembacaan metadata, atau akses <i>cache</i> memicu deteksi.	• <i>Alert</i> atau <i>warning</i> saat enumerasi <i>folder</i> • Peringatan <i>suspicious behavior</i> • Jumlah <i>event</i> yang ditandai
Deteksi trafik eksfiltrasi	Mengamati apakah <i>batch</i> JSON yang dikirim melalui HTTPS dianggap anomali oleh <i>anti-spyware</i> .	• <i>Alert</i> jaringan • <i>Block/allow</i> status • Jumlah <i>request</i> yang ditandai
Keberhasilan eksfiltrasi	Mengukur seberapa banyak <i>batch</i> yang berhasil dikirim dan diterima oleh server.	• <i>Success rate</i> (%) • Jumlah <i>batch</i> gagal • <i>Latency</i> per <i>batch</i>
Stabilitas modul selama eksekusi	Menilai apakah modul berjalan stabil tanpa mengalami <i>crash</i> atau <i>error</i> selama skenario uji.	• <i>Crash count</i> • <i>Error log</i> • Durasi eksekusi tanpa gangguan

V.2.2 Kriteria Keberhasilan

Kriteria keberhasilan ditetapkan untuk menentukan apakah modul *mission* dan modul *exfiltration* dapat beroperasi sesuai tujuan penelitian serta mengevaluasi respon produk keamanan terhadap aktivitas tersebut. Pengujian dinyatakan berhasil apabila memenuhi kriteria keberhasilan pada Tabel V.4.

Tabel V.4 Kriteria Keberhasilan

Aspek yang Dievaluasi	Kriteria Keberhasilan
Deteksi pada tingkat proses	- Jumlah <i>alert</i> proses ≤ 5 - Tidak ada <i>block</i> total oleh <i>anti-spyware</i>
Deteksi aktivitas <i>in-memory/fileless</i>	- Tidak muncul <i>alert High Severity</i> <i>Alert medium/low</i> ≤ 2 kali
Deteksi pada aktivitas <i>crawling</i>	<i>Alert crawling</i> ≤ 2 kali - Tidak ada pemblokiran permanen pada akses direktori
Deteksi trafik eksfiltrasi	<i>Alert jaringan</i> ≤ 2 kali - Tidak terjadi <i>connection block</i> - Minimal 70% <i>request</i> HTTPS sukses
Keberhasilan eksfiltrasi	<i>Success rate</i> $\geq 80\%$ <i>Batch</i> gagal $\leq 20\%$ - Latensi per <i>batch</i> masih dalam rentang wajar untuk jaringan lokal (≤ 5 detik)
Stabilitas modul selama eksekusi	- Tidak ada <i>crash</i> fatal - Modul tetap berjalan sampai skenario uji selesai

V.3 Analisis Risiko

Analisis risiko diperlukan untuk mengidentifikasi potensi masalah yang dapat muncul selama implementasi dan pengujian modul *mission* serta modul *exfiltration*. Langkah mitigasi disusun agar setiap risiko dapat diminimalkan.

Tabel V.5 Analisis Risiko dan Mitigasi

Risiko	Penyebab	Dampak	Mitigasi
Kegagalan eksfiltrasi	Server lokal lambat merespon, koneksi terhambat <i>firewall</i> , atau interval pengiriman terlalu rapat	Data <i>batch</i> hilang atau tidak tercatat	Menambah interval acak, memvalidasi <i>end-point</i> sebelum uji, dan menyesuaikan ukuran <i>batch</i>
<i>Crash</i> atau <i>hang</i> pada modul	Modul <i>mission</i> atau eksfiltrasi mengalami <i>error runtime</i>	Skenario uji tidak dapat diselesaikan	Melakukan uji coba awal, <i>logging</i> sederhana, dan pemantauan penggunaan memori selama eksekusi
Ketidaksesuaian data	Kesalahan saat <i>formatting</i> JSON atau data <i>buffer</i> tidak bersih	<i>Batch</i> tidak diterima server atau <i>parsing</i> gagal	Validasi JSON sebelum pengiriman, gunakan struktur <i>payload</i> minimalis
Gangguan pada <i>host</i> Windows selama pengujian	Beban <i>crawling</i> atau proses eksfiltrasi memicu <i>throttling</i> CPU-/memori	Latensi meningkat dan hasil uji bias	Membatasi intensitas <i>crawling</i> , memastikan <i>host</i> dalam kondisi <i>idle</i> saat uji

DAFTAR PUSTAKA

- Abualhaj, Mosleh, Ahmad Al-Shamayleh, Alhamza Munther, Sumaya Alkhatib, Mohammad Hiari, dan Mohammed Anbar. 2024. "Enhancing spyware detection by utilizing decision trees with hyperparameter optimization". Accessed November 20, 2025, *Bulletin of Electrical Engineering and Informatics* 13 (5). <https://doi.org/10.11591/eei.v13i5.7939>.
- Baldota, Varun, Radhika Shelar, dan Pranali Parbhane. 2025. "Spyware Detection: A Comprehensive Review". Accessed December 1, 2025, *International Journal on Science and Technology (IJSAT)*.
- Blessing, Lucienne T. M., dan Amaresh Chakrabarti. 2009. *DRM, a Design Research Methodology*. Accessed December 2, 2025. London: Springer. ISBN: 978-1-84882-586-4. <https://doi.org/10.1007/978-1-84882-587-1>.
- Elghaly, Yehia. 2024. "Stealth in Plain Sight: The Hidden Threat of PowerShell Fileless Malware and Its Evasion of Modern EDRs & AVs". Accessed November 20, 2025, *Research Square*, <https://doi.org/10.21203/rs.3.rs-5248527/v1>.
- Jegan, R., V. Rajavarman, dan S. Geetha. 2024. "AI-Enhanced Dark Web Crawler for Cybersecurity Monitoring". Accessed December 1, 2025, *Journal of Propulsion Technology*.
- Kareem, Karwan. 2024. "A Comprehensive Analysis of Pegasus Spyware and Its Implications for Digital Privacy and Security". Accessed November 20, 2025, *International Journal Intelligent Systems and Applications in Engineering*, <https://ijisae.org/index.php/IJISAE/article/view/5527>.
- King, James, Gueltoum Bendiab, Nick Savage, dan Stavros Shiaeles. 2021. "Data Exfiltration: Methods and Detection Countermeasures". Accessed November 20, 2025, *IEEE*, <https://doi.org/10.1109/CSR51186.2021.9527962>.

- Peng, Jiaxin, Chun Guo, Yuan Ping, Yunhe Cui, Yi Chen, dan Shen Guowei. 2024. "SNDMI: Spyware network traffic detection method based on inducement operations". Accessed December 1, 2025, *Elsevier* 140. <https://doi.org/10.1016/j.cose.2024.103806>.
- Qabalin, Majdi, Muawya Naser, dan Mouhammd Alkasassbeh. 2022. "Android Spyware Detection Using Machine Learning: A Novel Dataset". Accessed November 20, 2025, *Sensors*, <https://doi.org/10.3390/s22155765>.
- Reddy, Reddyvary, M. Rao, Singam Reddy, Kota Redd, dan Banoth Nayak. 2024. "A Review on Spyware Creation and Detection". Accessed November 20, 2025, *International Journal of Engineering Research & Technology (IJERT)*, <https://doi.org/10.17577/IJERTV13IS030140>.
- Salih, Huda, dan Mustafa Mohammed. 2020. "Spyware Injection in Android using Fake Application". Accessed November 20, 2025, *International Conference on Computer Science and Software Engineering (CSASE)*.
- Sudhakar dan Sushil Kumar. 2020. "An emerging threat Fileless malware: a survey and research challenges". Accessed December 1, 2025, *Cybersecurity*, <https://doi.org/10.1186/s42400-019-0043-x>.
- Ullah, Faheem, Matthew Edwards, Rajiv Ramdhany, Ruzanna Chitchyan, M. Babar, dan Awais Rashid. 2018. "Data exfiltration: A review of external attack vectors and countermeasures". Accessed December 1, 2025, *Elsevier* 101. <https://doi.org/10.1016/j.jnca.2017.10.016>.
- Verma, Varun, Saurabh Dubey, Tajwar Khan, Prem, dan Himanshu Pandey. 2021. "Cyber Security: The Study on Information Gathering". Accessed November 20, 2025, *International Journal of Scientific Research and Engineering Trends (IJSRET)*, ISSN: 2395-566X.
- Wang, Siyu, Haiyong Wu, Ning Lu, Wenbo Shi, dan Liu Zhiquan. 2025. "ATSDetector: An Android Trojan spyware detection approach with multi-features". Accessed December 1, 2025, *Elsevier* 150. <https://doi.org/10.1016/j.cose.2024.104219>.
- Zhan, Mengqhi, Yang Li, Guangxi Yu, Bo Li, dan Weiping Wang. 2022. "Detecting DNS over HTTPS based data exfiltration". Accessed December 1, 2025, *Elsevier* 140. <https://doi.org/10.1016/j.cose.2024.103806>.